

codex-windows / 019f2174-396f-71a1-99ab-269083c0eccf

Metadata

```
- Source: `codex-windows`
- Kind: `codex`
- Source file: `C:\Users\avidu\codex\sessions\2026\07\02\rollout-2026-07-02T11-41-32-019f2174-396f-71a1-99ab-269083c0eccf.jsonl`
- SHA-256: `6952bdf4851f333f1a74c7852287f86b52e1904f713061f88172c9d5ec1f3f24`
- Source modified: `2026-07-02T06:11:32+00:00`
- Imported at: `2026-07-05T16:48:15+00:00`
- cli_version: `0.142.5`
- cwd: `C:\Users\avidu\Projects\badminton-highlight-indexer`
- model_provider: `openai`
- session_id: `019f2174-396f-71a1-99ab-269083c0eccf`
- source: `vscode`
```

Transcript

1. user (2026-07-02T06:11:32.849Z)

In backend/worker/__main__.py, `cmd\_train` cannot match labels to videos when labels use the new canonical GCS naming `labels/<YYYY-MM-DD>\_<name>.rallies.csv` (see docs/DATA_IN_GCS.md §7.1 + §7b blocker #1).

Root cause (verified): cmd_train derives the per-clip key as `name = fname.replace(".rallies.csv", "").replace(".csv", "")` (backend/worker/__main__.py:384), e.g. `2026-06-22\_GX020094`. It then looks up the video via `video\_by\_stem.get(name)` (:387), where `video\_by\_stem` is keyed by the bare video stem `os.path.splitext(vname)[0]` = `GX020094` (:378). The date prefix means the stems never match ? every clip hits the "no video for ... skipping" branch (:389) ? `worker train --trajectory-source detector --corpus-uri gcs://khealsutra-rally-corpus` aborts with "<2 videos". (synth-train + all eval are unaffected ? they don't join videos.)

Fix: strip a leading ISO date prefix when deriving the clip name, e.g. after line 384 add `name = re.sub(r"^\d{4}-\d{2}-\d{2}\_", "", name)` (import re). Better: factor the label?clip-name derivation into a small helper and unit-test it (a dated label name and a bare label name both map to the bare clip stem, and join to `videos/<stem>.mp4`). Confirm `import re` and that the spec `name` used downstream is the clean (date-stripped) name.

Verify: run `python run\_all\_tests.py --cov=backend --cov=training --cov-fail-under=70` + `ruff check .` + `mypy backend training` (the repo's pre-LGTM gate). Branch off origin/master (Khealsutra/badminton-highlight-indexer), one PR for this item only. The cloud corpus to test against: gs://khealsutra-rally-corpus (labels/<date>_<name>.rallies.csv + videos/<name>.mp4; only videos/mahadevpura_1.mp4 is uploaded so far).

2. assistant (2026-07-02T06:11:32.849Z)

I'll start by following the repo conventions: pull latest, read the target code, then branch off `origin/master` explicitly. Let me gather context in parallel.

3. assistant (2026-07-02T06:11:32.849Z)

```
[external_agent_tool_call: Read]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\backend\worker\__main__.py
[/external_agent_tool_call]
```

4. assistant (2026-07-02T06:11:32.849Z)

```
[external_agent_tool_result]
1      """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3      python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
```

```

4          [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5          python -m backend.worker train ...          # Gen-0 train-from-bucket (follow-up)
6
7          The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
``local:``/``
8          is an identity passthrough; ``s3:``/`` / ``gcs:``/`` is an object store), so existing
local runs
9          are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
10         ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
key) ?
11         the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
12
13         This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
CLI uses
14         (validator registry ? segmenter ? report), never a forked pipeline.
15         """
16         from __future__ import annotations
17
18         import argparse
19         import csv
20         import json
21         import logging
22         import os
23         import sys
24         import tempfile
25         from typing import Any, List
26
27         from backend import storage
28         from backend.compute import ComputeJobSpec, get_compute_backend
29         from backend.config import (
30             StartupCheckError,
31             enforce_startup_checks,
32             load_config,
33             probe_cuda_available,
34         )
35         from backend.infrastructure.database import Database
36         from backend.pipeline.segmenters.base import segmenter_registry
37         from backend.pipeline.validators.base import registry as validator_registry
38         from backend.reporting import emit_indexer_report
39         from backend.results import Failure
40         from backend.utils.ffmpeg import ffprobe_bin # P2a: single ffmpeg/ffprobe resolver
41         from backend.utils.hashing import compute_video_id
42
43         logger = logging.getLogger("backend.worker")
44
45
46         def _coerce(v: str) -> Any:
47             """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
str."""
48             low = v.lower()
49             if low in ("true", "false"):
50                 return low == "true"
51             for cast in (int, float):
52                 try:
53                     return cast(v)
54                 except ValueError:
55                     pass
56             return v
57
58
59         def _apply_overrides(config: Any, sets: List[str]) -> None:
60             """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
61             for kv in sets or []:
62                 key, sep, val = kv.partition("=")

```

```

63         if not sep:
64             raise ValueError(f"--set expects k=v, got {kv!r}")
65         parts = key.split(".")
66         obj = config
67         for p in parts[:-1]:
68             obj = getattr(obj, p)
69         setattr(obj, parts[-1], _coerce(val))
70
71
72     def _write_intervals_csv(segments: List[dict], path: str) -> None:
73         """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
friendly
74         artifact (same schema as the rally-annotator labels)."""
75         with open(path, "w", newline="") as f:
76             w = csv.writer(f)
77             w.writerow(["start", "end", "shot_count", "ending_reason"])
78             for s in segments:
79                 w.writerow([
80                     f'{float(s.get("start_time", 0.0)):.3f}',
81                     f'{float(s.get("end_time", 0.0)):.3f}',
82                     s.get("shot_count", ""),
83                     s.get("ending_reason", s.get("shot_count_confidence", "")),
84                 ])
85
86
87     def _write_report_json(video_id: str, segments: List[dict], status: str, path: str) ->
None:
88         with open(path, "w") as f:
89             json.dump({
90                 "video_id": video_id,
91                 "status": status,
92                 "segment_count": len(segments),
93                 "segments": [{"start": float(s.get("start_time", 0.0)),
94                             "end": float(s.get("end_time", 0.0))} for s in segments],
95             }, f, indent=2)
96
97
98     def _run_startup_checks(config: Any) -> bo...
[/external_agent_tool_result]

```

5. assistant (2026-07-02T06:11:32.849Z)

Now let me pull latest, check the docs referenced, and look at existing worker tests.

6. assistant (2026-07-02T06:11:32.849Z)

```

[external_agent_tool_call: Bash]
description: Fetch origin and show status
command: git fetch origin 2>&1 | tail -5 && echo "---STATUS---" && git status -sb 2>&1 | head -5
[/external_agent_tool_call]

```

7. assistant (2026-07-02T06:11:32.849Z)

```

[external_agent_tool_call: Read]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\docs\DATA_IN_GCS.md
[/external_agent_tool_call]

```

8. assistant (2026-07-02T06:11:32.849Z)

```

[external_agent_tool_result]
---STATUS---

```

work/golden-corpus-hygiene...origin/work/golden-corpus-hygiene

?? artifacts/

?? docs/KHELSUTRA_PRODUCT_OVERVIEW.md

[/external_agent_tool_result]

9. assistant (2026-07-02T06:11:32.849Z)

[external_agent_tool_result]

```
1      # Data in GCS ? the corpus source-of-truth map
2
3      **Status:** LIVE / in-progress (migration not complete). **Owner-gated** decisions in
$7.
4      **Last inventoried:** 2026-06-22 against `gs://khelsutra-rally-corpus`.
5      **Goal (owner, 2026-06-22):** run **all** training + inference from the cloud with **all
data in GCS**,
6      so the local Windows box (and the intermittently-mounted external drive) is **no longer
a blocker**.
7
8      > **? Other sessions: this is where the data lives.** Pull from `gs://khelsutra-rally-
corpus`, not
9      > from anyone's `C:` / `F:` / Google Drive. If something you need isn't in the bucket
yet, it's in the
10     > **gap table ($5)** ? flag it / migrate it, don't hard-code a local path.
11
12     This doc is the **data map**. The *machine* runbooks are
[ `SETUP_NEW_MACHINE.md` ](SETUP_NEW_MACHINE.md)
13     (spin up a box) and [ `deploy/gcp/README.md` ](../deploy/gcp/README.md) (provision GPU +
train/infer).
14     The cloud engine itself was built by the (archived) DECOUPLED_COMPUTE project
15     (`docs/archives/past_projects/DECOUPLED_COMPUTE/`); this doc does **not** redesign it ?
it records
16     *where the bytes are* and *what's still missing*.
17
18     ---
19
20     ## 1. The bucket
21
22     | | |
23     |---|---|
24     | Bucket | **gs://khelsutra-rally-corpus** (single corpus bucket) |
25     | Project | `gen-lang-client-0306026826` (billing *Khelsutra Billing*) |
26     | Region | `asia-south1` (Mumbai), UBLA, co-located with the GPU VMs |
27     | Auth | SA `rally-worker@gen-lang-client-0306026826.iam.gserviceaccount.com`
(`roles/storage.objectAdmin`); key at `~/gcp/rally-worker-sa.json` (local only, **never
committed**). Interactive: `gcloud auth login`. |
28     | Size today | ~2.06 GB (mostly past dry-run reels/outputs ? **not** the full corpus) |
29
30     The engine is **backend-agnostic by URI** (`backend/storage/ref.py`): `gs://` (or
`gs:///`),
31     `s3:///`, `gdrive:///`, or a bare local path. Pick the backend with **config + creds, no
code change**.
32
33     ---
34
35     ## 2. Canonical layout (the target contract)
36
37     The cloud worker's input/output contract (`backend/worker/__main__.py`; lineage:
`DECOUPLED_COMPUTE/04`):
38
39
40     | Prefix | Holds | Who reads/writes it |
41     |---|---|---|
42     | `videos/` | source videos / proxies (`<clip>.mp4`) | **input** to `worker infer` &
`worker train` |
43     | `labels/` | golden rally labels (`<clip>.rallies.csv`) | **input** ground truth to
`worker train` + all eval |
44     | `trajectories/` | cached WASB trajectory CSVs | ? **eval/litmus input** (GPU-free),
consumed via `manifests/golden.json`. **NOT** read by `worker train` (it synths or re-extracts)
|
```

```

45 | `weights/<version>/` | trained Gen-0 bundles (`weights.json` + a run `manifest.json`)
| **output** of `worker train` |
46 | `models/` | upstream pretrained weights (`wasb_badminton_best.pth.tar`) | **staged to
a local path** (`$WASB_WEIGHTS` / `~/models/?`) for the detector ? not read from `gs://` in
place |
47 | `outputs/<video_id>/` | per-video inference outputs (intervals, etc.) | **output** of
`worker infer` |
48 | `highlights/`, `promo/` | rendered reels + promo artifacts | output / demo |
49
50 > **Note on the manifest.** `worker train` does **not** read a pre-staged corpus
`manifest.json` ? it
51 > *builds* one on-box at train time from `labels/` (+`videos/`), extracting trajectories
**synthetically
52 > by default** (`--trajectory-source synth`, CPU-mock) or for real via opt-in
`--trajectory-source
53 > detector` (needs `videos/`). A `manifest.json` only appears as an **output** inside
each
54 > `weights/<version>/` bundle. The **eval/litmus** path is what wants a stable, path-
resolvable manifest
55 > (§7.3) ? and `manifests/golden.json` is currently consumed by **no code yet** (eval is
local-FS only;
56 > see Known blockers).
57
58 ---
59
60 ## 3. What's actually in the bucket today (2026-06-22)
61
62 > **Phase-1 landed 2026-06-22:** added `trajectories/` (15), 15 canonical
`labels/<date>_<name>.rallies.csv`
63 > (= original-6 + `GX010137`/...
[/external_agent_tool_result]

```

10. assistant (2026-07-02T06:11:32.849Z)

The doc confirms the bug precisely. Now let me find the existing worker tests to match conventions.

11. assistant (2026-07-02T06:11:32.849Z)

```

[external_agent_tool_call: Bash]
description: Show origin/master head and branch divergence
command: git log --oneline origin/master -3 2>&1 && echo "---" && git log --oneline
origin/master..origin/work/golden-corpus-hygiene 2>&1 | head -20
[/external_agent_tool_call]

```

12. assistant (2026-07-02T06:11:32.849Z)

```

[external_agent_tool_result]
d301a7f fix(nightly): GCP-L4-validated fixes ? gs:// fetch dst, WASB env, weights path, pure-
engine (#318)
fa2342c feat(nightly): E2E engine reel-count regression on a GCP GPU VM (emailed) + cost (#315)
7a40434 feat(eval): golden fixtures P3 ? 6 curated, trimmed, de-attributed REAL Tier-0 fixtures
+ --max-frames trim (#317)
---
1b33f55 docs(data): correct DATA_IN_GCS per adversarial review (blockers + accuracy)
53467c9 docs(data): record stale-label cleanup + set up Phase-2 video copy
6ae3503 docs(data): record Phase-1 GCS migration (trajectories + labels + manifest)
8884c9f docs(data): add DATA_IN_GCS.md ? the corpus source-of-truth map (GCS)
[/external_agent_tool_result]

```

13. assistant (2026-07-02T06:11:32.849Z)

```

[external_agent_tool_call: Glob]
input: {"pattern":"**/test*worker*.py"}

```

```
[/external_agent_tool_call]
```

14. assistant (2026-07-02T06:11:32.849Z)

```
[external_agent_tool_result]
.claude\worktrees\bhi-execpolicy\tests\test_job_worker_loop.py
.claude\worktrees\agent-a771ff1d85e18f2ed\tests\test_job_worker_loop.py
.claude\worktrees\tier1-windowing\tests\test_job_worker_loop.py
.claude\worktrees\w2-safeguard\tests\test_job_worker_loop.py
.claude\worktrees\hardchop\tests\test_job_worker_loop.py
.claude\worktrees\boxhill17\tests\test_job_worker_loop.py
.claude\worktrees\pr-stitch\tests\test_job_worker_loop.py
.claude\worktrees\gap-doc\tests\test_job_worker_loop.py
.claude\worktrees\resweep\tests\test_job_worker_loop.py
.claude\worktrees\recdoc\tests\test_job_worker_loop.py
tests\test_job_worker_module_seam.py
tests\test_worker.py
tests\test_job_worker_loop.py
[/external_agent_tool_result]
```

15. assistant (2026-07-02T06:11:32.849Z)

```
[external_agent_tool_call: Grep]
input: {"pattern":"cmd_train|video_by_stem|trajectory_source|_VIDEO_EXTS", "output_mode":"files_w
ith_matches"}
[/external_agent_tool_call]
```

16. assistant (2026-07-02T06:11:32.849Z)

```
[external_agent_tool_result]
Found 7 files
docs\DATA_IN_GCS.md
backend\worker\__main__.py
docs\COMPUTE_DECOUPLED_SERVING\README.md
docs\PER_VIDEO_WORKSPACE.md
docs\DESKTOP_APP_PLAN.md
tests\test_worker.py
tools\generate_highlights.py
[/external_agent_tool_result]
```

17. assistant (2026-07-02T06:11:32.849Z)

`tests/test\_worker.py` is the home for these tests. Let me read it, focusing on the train-related tests.

18. assistant (2026-07-02T06:11:32.849Z)

```
[external_agent_tool_call: Read]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_worker.py
[/external_agent_tool_call]
```

19. assistant (2026-07-02T06:11:32.850Z)

```
[external_agent_tool_result]
1      """Worker dispatcher: pull (storage) ? run pipeline ? push (storage).
2
3      `cmd_infer` is exercised end-to-end over the LOCAL storage backend with the validator +
4      segmenter mocked, so it needs no real video/GPU/network. The real S3 round-trip is
validated
5      on the droplet against MinIO.
6      """
7      import json
8
9      import pytest
```

```

10
11     from backend.config import load_config
12     from backend.worker import __main__ as wmain
13
14
15     # ----- pure helpers
16
17     @pytest.mark.parametrize("raw,expected", [
18         ("true", True), ("False", False), ("3", 3), ("2.5", 2.5), ("wasb_hybrid",
19         "wasb_hybrid"),
20     ])
21     def test_coerce(raw, expected):
22         assert wmain._coerce(raw) == expected and type(wmain._coerce(raw)) is type(expected)
23
24     def test_apply_overrides_on_typed_config():
25         cfg = _fresh_config()
26         wmain._apply_overrides(cfg, ["indexing.skip_ai_handoff=true",
27         "indexing.min_segment_duration=1.5",
28         "sport=tennis"])
29         assert cfg.indexing.skip_ai_handoff is True
30         assert cfg.indexing.min_segment_duration == 1.5
31         assert cfg.sport == "tennis"
32
33     def test_apply_overrides_rejects_bad_format():
34         cfg = _fresh_config()
35         with pytest.raises(ValueError):
36             wmain._apply_overrides(cfg, ["no_equals_sign"])
37
38
39     def test_write_intervals_csv(tmp_path):
40         segs = [{"start_time": 2.0, "end_time": 5.0, "shot_count": 4,
41         "shot_count_confidence": "LocalNoAI"},
42         {"start_time": 9.0, "end_time": 12.5}]
43         out = tmp_path / "intervals.csv"
44         wmain._write_intervals_csv(segs, str(out))
45         lines = out.read_text().splitlines()
46         assert lines[0] == "start,end,shot_count,ending_reason"
47         assert lines[1].startswith("2.000,5.000,4,")
48         assert lines[2].startswith("9.000,12.500,")
49
50     def test_write_report_json(tmp_path):
51         out = tmp_path / "report.json"
52         wmain._write_report_json("vid1", [{"start_time": 1.0, "end_time": 2.0}], "success",
53         str(out))
54         data = json.loads(out.read_text())
55         assert data["video_id"] == "vid1" and data["segment_count"] == 1
56         assert data["segments"][0] == {"start": 1.0, "end": 2.0}
57
58     def test_parser_infer_accepts_set_and_uris():
59         args = wmain.build_parser().parse_args([
60             "infer", "--video-uri", "s3://b/v.mp4", "--out-uri", "s3://b",
61             "--set", "indexing.skip_ai_handoff=true", "--set", "sport=tennis"])
62         assert args.cmd == "infer" and args.video_uri == "s3://b/v.mp4"
63         assert args.set == ["indexing.skip_ai_handoff=true", "sport=tennis"]
64
65
66     # ----- cmd_infer (local,
67     mocked)
68
69     class _OKResult:
70         passed = True

```

```

70         validator_name = "fake"
71         message = "ok"
72
73         def model_dump(self):
74             return {"validator_name": "fake", "passed": True, "message": "ok"}
75
76
77     class _FailResult(_OKResult):
78         passed = False
79         message = "too blurry"
80
81
82     class _FakeSeg:
83         def __init__(self, db, config):
84             pass
85
86         def process_video(self, video_id, path, max_frames=None):
87             return [
88                 {"start_time": 2.0, "end_time": 5.0, "shot_count": 4,
"shot_count_confidence": "LocalNoAI"},
89                 {"start_time": 9.0, "end_time": 12.0, "shot_count": 6,
"shot_count_confidence": "LocalNoAI"},
90             ]
91
92
93     def _fresh_config():
94         # Load defaults without touching any cwd config.json.
95         import tempfile
96         p = tempfile.NamedTemporaryFile("w", suffix=".json", delete=False)
97         p.write("{}")
98         p.close()
99         return load_config(p.name)
100
101
102     @pytest.fixture
103     def mocked_pipeline(monkeypatch):
104         monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vid123")
105         monkeypatch.setattr(wmain.validator_registry, "run_all_with_context",
106                             lambda path, cfg: ([_OKResult(...
[/external_agent_tool_result]

```

20. assistant (2026-07-02T06:11:32.850Z)

I now have everything I need. The bug and fix are exactly as described in `docs/DATA\_IN\_GCS.md` \$7b blocker #1. Let me create the branch off `origin/master` (per branching-safety convention), then apply the fix.

21. assistant (2026-07-02T06:11:32.850Z)

```

[external_agent_tool_call: Bash]
description: Branch off origin/master explicitly
command: git stash list 2>&1 | head -3; echo "---untracked that matter---"; git status
--porcelain 2>&1 | head; echo "---create branch from origin/master---"; git checkout -b
fix/worker-train-dated-label-join origin/master 2>&1 | tail -5; echo "---verify HEAD---"; git
branch --show-current; git log --oneline -1
[/external_agent_tool_call]

```

22. assistant (2026-07-02T06:11:32.850Z)

```

[external_agent_tool_result]
---untracked that matter---
?? artifacts/
?? docs/KHELSUTRA_PRODUCT_OVERVIEW.md
---create branch from origin/master---

```

```
branch 'fix/worker-train-dated-label-join' set up to track 'origin/master'.
Switched to a new branch 'fix/worker-train-dated-label-join'
---verify HEAD---
fix/worker-train-dated-label-join
d301a7f fix(nightly): GCP-L4-validated fixes ? gs:// fetch dst, WASB env, weights path, pure-
engine (#318)
[/external_agent_tool_result]
```

23. assistant (2026-07-02T06:11:32.850Z)

Now I'm on a clean branch off `origin/master`. Let me apply the fix. First, add `import re`:

24. assistant (2026-07-02T06:11:32.850Z)

```
[external_agent_tool_call: Edit]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\backend\worker\__main__.py
[/external_agent_tool_call]
```

25. assistant (2026-07-02T06:11:32.850Z)

```
[external_agent_tool_result]
The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\worker\__main__.py has been
updated successfully. (file state is current in your context ? no need to Read it back)
[/external_agent_tool_result]
```

26. assistant (2026-07-02T06:11:32.850Z)

Now add the helper near `\_VIDEO\_EXTS`:

27. assistant (2026-07-02T06:11:32.850Z)

```
[external_agent_tool_call: Edit]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\backend\worker\__main__.py
[/external_agent_tool_call]
```

28. assistant (2026-07-02T06:11:32.850Z)

```
[external_agent_tool_result]
The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\worker\__main__.py has been
updated successfully. (file state is current in your context ? no need to Read it back)
[/external_agent_tool_result]
```

29. assistant (2026-07-02T06:11:32.850Z)

Now wire the helper into `cmd\_train`:

30. assistant (2026-07-02T06:11:32.850Z)

```
[external_agent_tool_call: Edit]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\backend\worker\__main__.py
[/external_agent_tool_call]
```

31. assistant (2026-07-02T06:11:32.850Z)

```
[external_agent_tool_result]
The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\worker\__main__.py has been
updated successfully. (file state is current in your context ? no need to Read it back)
[/external_agent_tool_result]
```

32. assistant (2026-07-02T06:11:32.850Z)

Now add the unit tests for the helper plus a regression test for the dated-label?bare-video join. Let me insert them after the existing train tests:

33. assistant (2026-07-02T06:11:32.850Z)

[external_agent_tool_call: Edit]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_worker.py
[/external_agent_tool_call]

34. assistant (2026-07-02T06:11:32.850Z)

[external_agent_tool_result]
The file C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_worker.py has been updated successfully. (file state is current in your context ? no need to Read it back)
[/external_agent_tool_result]

35. assistant (2026-07-02T06:11:32.850Z)

Now let me run the targeted worker tests first, then the full pre-LGTM gate.

36. assistant (2026-07-02T06:11:32.850Z)

[external_agent_tool_call: Bash]
description: Run worker tests
command: python -m pytest tests/test_worker.py -q 2>&1 | tail -25
[/external_agent_tool_call]

37. assistant (2026-07-02T06:11:32.850Z)

[external_agent_tool_result]
..... [100%]
26 passed in 2.16s
[/external_agent_tool_result]

38. assistant (2026-07-02T06:11:32.850Z)

All 26 worker tests pass. Now the full pre-LGTM gate ? tests with coverage, ruff, and mypy:

39. assistant (2026-07-02T06:11:32.850Z)

[external_agent_tool_call: Bash]
description: Run full test suite with coverage
command: python run_all_tests.py --cov=backend --cov=training --cov-fail-under=70 2>&1 | tail -40
[/external_agent_tool_call]

40. assistant (2026-07-02T06:11:32.850Z)

[external_agent_tool_result]
backend\results.py 27 0 100%
backend\sports__init__.py 17 0 100%
backend\sports\badminton.py 13 1 92%
backend\sports\base.py 12 3 75%
backend\storage__init__.py 57 4 93%
backend\storage\base.py 14 0 100%
backend\storage\gcs.py 41 4 90%
backend\storage\gdrive.py 61 3 95%
backend\storage\local.py 49 3 94%
backend\storage\ref.py 48 0 100%
backend\storage\s3.py 47 5 89%
backend\tools__init__.py 0 0 100%
backend\tools\export.py 75 2 97%
backend\tools\rally_slicer.py 105 47 55%
backend\utils__init__.py 0 0 100%
backend\utils\app_home.py 40 0 100%
backend\utils\ffmpeg.py 24 0 100%
backend\utils\geometry.py 29 0 100%

backend\utils\hashing.py	9	0	100%
backend\utils\logging_setup.py	11	0	100%
backend\utils\paths.py	33	3	91%
backend\utils\redact.py	16	0	100%
backend\utils\run_telemetry.py	126	12	90%
backend\utils\single_instance.py	43	11	74%
backend\utils\video.py	26	3	88%
backend\utils\workspace.py	49	0	100%
backend\worker__init__.py	0	0	100%
backend\worker__main__.py	300	37	88%
training__init__.py	0	0	100%
training\gen0__init__.py	0	0	100%
training\gen0\harness.py	129	9	93%

TOTAL	10001	1449	86%
-------	-------	------	-----

Required test coverage of 70% reached. Total coverage: 85.51%

===== short test summary info =====

FAILED tests/test_served_gate_a_regression.py::test_proposal_windowing_gate_a_is_f1_neutral_and_cuts_over_seg

1 failed, 1426 passed, 7 warnings in 152.62s (0:02:32)

=====

RESULT: FAIL - pytest exit 1

=====

[/external_agent_tool_result]

41. assistant (2026-07-02T06:11:32.850Z)

One unrelated test failed: `test\_served\_gate\_a\_regression.py` (gate-A proposal windowing) ? a code path my change doesn't touch. I must confirm it's pre-existing, not caused by me. Let me investigate.

42. assistant (2026-07-02T06:11:32.851Z)

[external_agent_tool_call: Bash]

description: Re-run the failing test in isolation

command: python -m pytest "tests/test_served_gate_a_regression.py::test_proposal_windowing_gate_a_is_f1_neutral_and_cuts_over_seg" -q 2>&1 | tail -30

[/external_agent_tool_call]

43. assistant (2026-07-02T06:11:32.851Z)

[external_agent_tool_result]

```

iv = [(w["start"], w["end"]) for w in wins]
est = rally_gate.estimate_play_axis(pts)
gated = rally_gate.gate_windows(pts, iv, fps, min_crossings=0, estimate=est)
on = [(g.start, g.end) for g in gated if g.crossings >= 2]
gts = load_golden_gts(s["labels"])
d_on.append(score(on, gts).f1)
d_off.append(score(iv, gts).f1)
seg_on.append(_segm.segment_count_ratio(on, gts))
seg_off.append(_segm.segment_count_ratio(iv, gts))

```

n = len(d_on)

mean_delta = (sum(d_on) - sum(d_off)) / n

mean_seg_off, mean_seg_on = sum(seg_off) / n, sum(seg_on) / n

Durable direction-of-effect (corpus-independent): the gate only drops windows, so

under the

over-segmenting proposal windowing it lowers the over-seg ratio.

assert mean_seg_on < mean_seg_off, (mean_seg_on, mean_seg_off)

Honest measured snapshot (2026-06-18, present 9-clip multicourt set): mean ?F1 ?

-0.001,

over-seg 1.16 ? 0.64 (was +0.074 / 1.68 ? 1.01 on the retired original-6). ~F1-neutral

band:

assert mean_delta == pytest.approx(0.0, abs=0.03), mean_delta

> assert mean_seg_off == pytest.approx(1.16, abs=0.10), mean_seg_off

```
E      AssertionError: 1.3639692020429994
E      assert 1.3639692020429994 == 1.16 ± 0.1
E
E      comparison failed
E      Obtained: 1.3639692020429994
E      Expected: 1.16 ± 0.1
```

```
tests\test_served_gate_a_regression.py:179: AssertionError
===== short test summary info =====
FAILED tests/test_served_gate_a_regression.py::test_proposal_windowing_gate_a_is_f1_neutral_and_cuts_over_seg
1 failed in 6.17s
[/external_agent_tool_result]
```

44. assistant (2026-07-02T06:11:32.851Z)

This failure is a **corpus-snapshot drift**, not my change: it asserts `mean_seg_off == 1.16 ± 0.10` against a local golden fixture set and got `1.36`. My change only touches `cmd_train`'s label-name derivation ? it cannot affect `rally_gate.gate_windows` / `load_golden_gts`. Let me prove it's pre-existing by reverting my two files and re-running.

45. assistant (2026-07-02T06:11:32.851Z)

```
[external_agent_tool_call: Bash]
description: Confirm test fails on clean tree too
command: echo "=== stash my changes (revert to clean origin/master tree) ==="; git stash push --
backend/worker/__main__.py tests/test_worker.py 2>&1 | tail -3
echo "=== run the failing test on clean origin/master ==="; python -m pytest "tests/test_served_
gate_a_regression.py::test_proposal_windowing_gate_a_is_f1_neutral_and_cuts_over_seg" -q 2>&1 |
tail -8
echo "=== restore my changes ==="; git stash pop 2>&1 | tail -3
[/external_agent_tool_call]
```

46. assistant (2026-07-02T06:11:32.851Z)

```
[external_agent_tool_result]
=== stash my changes (revert to clean origin/master tree) ===
Saved working directory and index state WIP on fix/worker-train-dated-label-join: d301a7f
fix(nightly): GCP-L4-validated fixes ? gs:// fetch dst, WASB env, weights path, pure-engine
(#318)
=== run the failing test on clean origin/master ===
E      comparison failed
E      Obtained: 1.3639692020429994
E      Expected: 1.16 ± 0.1

tests\test_served_gate_a_regression.py:179: AssertionError
===== short test summary info =====
FAILED tests/test_served_gate_a_regression.py::test_proposal_windowing_gate_a_is_f1_neutral_and_cuts_over_seg
1 failed in 6.46s
=== restore my changes ===

no changes added to commit (use "git add" and/or "git commit -a")
Dropped refs/stash@{0} (054f3749f67e9bc5ac704d3fb6ff22b3851e393d)
[/external_agent_tool_result]
```

47. assistant (2026-07-02T06:11:32.851Z)

Confirmed pre-existing: the identical failure (`1.3639...`) reproduces on a clean `origin/master` tree with my changes reverted. It's local golden-corpus snapshot drift (the `golden-corpus-hygiene` label cleanup changed the fixture set) ? unrelated to and untouched by my change. Now ruff + mypy:

48. assistant (2026-07-02T06:11:32.851Z)

```
[external_agent_tool_call: Bash]
description: Run ruff and mypy
command: echo "=== ruff ==="; ruff check . 2>&1 | tail -15
echo "=== mypy ==="; mypy backend training 2>&1 | tail -20
[/external_agent_tool_call]
```

49. assistant (2026-07-02T06:11:32.851Z)

```
[external_agent_tool_result]
=== ruff ===
help: Rewrite `bold` as a `def`

E731 Do not assign a `lambda` expression, use a `def`
--> artifacts\landing\make_og.py:24:1
|
23 | bold = lambda s: font("segoeuib.ttf", s)
24 | reg = lambda s: font("segoeui.ttf", s)
    | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
25 |
26 | M = 80 # margin
    |
help: Rewrite `reg` as a `def`
```

```
Found 2 errors.
No fixes available (2 hidden fixes can be enabled with the `--unsafe-fixes` option).
=== mypy ===
backend\annotations\base.py:85: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
backend\sports\__init__.py:7: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs [annotation-unchecked]
backend\pipeline\validators\base.py:36: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
backend\providers\__init__.py:7: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
Success: no issues found in 136 source files
[/external_agent_tool_result]
```

50. assistant (2026-07-02T06:11:32.851Z)

mypy is clean. ruff's 2 errors are both in `artifacts/landing/make\_og.py` ? the ****untracked**** `artifacts/` dir (owner/sibling WIP, `?? artifacts/` in the initial status), not my files and not part of this PR. Let me confirm my changed files are ruff-clean and check whether `artifacts/` is even tracked/ignored.

51. assistant (2026-07-02T06:11:32.851Z)

```
[external_agent_tool_call: Bash]
description: Confirm my files clean; check artifacts tracking
command: echo "=== ruff on my changed files only ==="; ruff check backend/worker/__main__.py
tests/test_worker.py 2>&1 | tail -10
echo "=== is artifacts/ tracked? ==="; git ls-files artifacts/ | head; echo "(empty above =
untracked)"
echo "=== is artifacts/ gitignored? ==="; git check-ignore artifacts/landing/make_og.py 2>&1 ||
echo "NOT ignored"
[/external_agent_tool_call]
```

52. assistant (2026-07-02T06:11:32.851Z)

```
[external_agent_tool_result]
=== ruff on my changed files only ===
All checks passed!
=== is artifacts/ tracked? ===
(empty above = untracked)
=== is artifacts/ qitignored? ===
```

```
NOT ignored
[/external_agent_tool_result]
```

53. assistant (2026-07-02T06:11:32.851Z)

Both my changed files pass ruff cleanly. The 2 ruff errors are in `artifacts/landing/make\_og.py`
? untracked owner WIP that won't exist in CI (not committed), so CI's `ruff check .` is
unaffected. I won't touch that file (sibling/owner WIP).

Let me review the final diff before committing:

54. assistant (2026-07-02T06:11:32.851Z)

```
[external_agent_tool_call: Bash]
description: Show final diff
command: git diff backend/worker/__main__.py tests/test_worker.py
[/external_agent_tool_call]
```

55. assistant (2026-07-02T06:11:32.851Z)

```
[external_agent_tool_result]
diff --git a/backend/worker/__main__.py b/backend/worker/__main__.py
index 081d8f5..08d8d84 100644
--- a/backend/worker/__main__.py
+++ b/backend/worker/__main__.py
@@ -20,6 +20,7 @@ import csv
     import json
     import logging
     import os
+import re
     import sys
     import tempfile
     from typing import Any, List
@@ -264,6 +265,29 @@ def cmd_infer(args: argparse.Namespace) -> int:
     #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem collision).
     _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")

+    #: Leading ISO-date prefix on a canonical label name (`labels/<YYYY-MM-DD>_<name>.rallies.csv`,
+    #: docs/DATA_IN_GCS.md §7.1). It disambiguates recycled GoPro names by authoring date but is
+    NOT
+    #: part of the clip's join key, so it's stripped when deriving the videos/ stem.
+    _LABEL_DATE_PREFIX = re.compile(r"^\d{4}-\d{2}-\d{2}_")
+
+    +
+    +
+    +def _label_clip_name(fname: str) -> str:
+    +    """Derive the clip name (the `videos/<name>.<ext>` join key) from a label filename.
+    +
+    +    Strips both the `rallies.csv` / `.csv` suffix and an optional leading ISO-date prefix,
+    so a
+    +    canonical dated label and a legacy bare label resolve to the SAME bare clip stem:
+    +
+    +    * `2026-06-22_GX020094.rallies.csv` -> `GX020094` (canonical, date-stamped)
+    +    * `GX020094.rallies.csv` -> `GX020094` (legacy bare ? same stem)
+    +    * `GX020094_proxy.rallies.csv` -> `GX020094_proxy` (`_proxy` is part of the
+    name, kept)
+    +    * `2026-06-21_mahadevpura_1.rallies.csv` -> `mahadevpura_1` (underscores in <name>
+    survive)
+    +
+    +    Without the date strip, `--trajectory-source detector` can't match a dated label to its
+    bare
+    +    video stem, so every clip is skipped and train aborts with `<2 videos` (DATA_IN_GCS §7b
+    #1).
+    +    """
```

```

+     name = fname.replace(".rallies.csv", "").replace(".csv", "")
+     return _LABEL_DATE_PREFIX.sub("", name)
+
def _probe_video_fps_width(path: str):
    """Robust ffmpeg (fps, frame_width) for a real video. Returns None on ANY failure.
@@ -380,8 +404,8 @@ def cmd_train(args: argparse.Namespace) -> int:
    print(f"[worker] trajectory source: {args.trajectory_source}")
    specs: List[dict] = []
    for uri in label_uris:
-         fname = storage.parse_uri(uri).name                    # e.g.
GX020094_proxy.rallies.csv
-         name = fname.replace(".rallies.csv", "").replace(".csv", "")
+         fname = storage.parse_uri(uri).name                    # e.g.
2026-06-22_GX020094.rallies.csv
+         name = _label_clip_name(fname)                        # -> GX020094 (date prefix
stripped)
        local_label = storage.fetch(uri, os.path.join(labels_dir, fname), config=storage_cfg)
        if args.trajectory_source == "detector":
            vuri = video_by_stem.get(name)
diff --git a/tests/test_worker.py b/tests/test_worker.py
index 642e46a..c48fa02 100644
--- a/tests/test_worker.py
+++ b/tests/test_worker.py
@@ -191,6 +191,66 @@ def test_cmd_infer_validation_failure_returns_2(tmp_path, monkeypatch):

    # --- cmd_train: pull labels -> synth trajectories -> manifest -> harness -> push bundle ---

+@pytest.mark.parametrize("fname,expected", [
+    ("2026-06-22_GX020094.rallies.csv", "GX020094"),          # canonical dated label
+    (DATA_IN_GCS, $7.1)                                       # legacy bare label -> SAME
+    ("GX020094.rallies.csv", "GX020094"),                    # legacy bare label -> SAME
+    clip_stem
+    ("GX020094_proxy.rallies.csv", "GX020094_proxy"),        # _proxy is part of the name,
+    kept
+    ("2026-06-21_mahadevpura_1.rallies.csv", "mahadevpura_1"), # underscores in <name> survive
+    ("v1.csv", "v1"),                                         # bare .csv suffix
+    ("2026-06-22_GX020094.csv", "GX020094"),                 # dated + bare .csv
+])
+def test_label_clip_name_strips_date_prefix(fname, expected):
+    """A dated label and a bare label must both resolve to the bare videos/<name> stem."""
+    assert wmain._label_clip_name(fname) == expected
+
+
+def test_cmd_train_detecto...
[/external_agent_tool_result]

```

56. assistant (2026-07-02T06:11:32.851Z)

The diff is clean. Let me verify the new regression test actually **catches** the bug ? revert only the source fix (keep the new test) and confirm the test fails without the fix:

57. assistant (2026-07-02T06:11:32.851Z)

```

[external_agent_tool_call: Bash]
description: Confirm regression test fails without fix
command: echo "=== revert ONLY the source fix, keep the new test ==="; git stash push --
backend/worker/_main__.py 2>&1 | tail -2
echo "=== run new regression test against UNFIXED source (expect FAIL) ==="; python -m pytest
"tests/test_worker.py::test_cmd_train_detector_joins_dated_labels_to_bare_videos"
"tests/test_worker.py::test_label_clip_name_strips_date_prefix" -q 2>&1 | tail -12
echo "=== restore the fix ==="; git stash pop 2>&1 | tail -2
[/external_agent_tool_call]

```

58. assistant (2026-07-02T06:11:32.851Z)

```
[external_agent_tool_result]
=== revert ONLY the source fix, keep the new test ===
Saved working directory and index state WIP on fix/worker-train-dated-label-join: d301a7f
fix(nightly): GCP-L4-validated fixes ? gs:// fetch dst, WASB env, weights path, pure-engine
(#318)
=== run new regression test against UNFIXED source (expect FAIL) ===
E      AttributeError: module 'backend.worker.__main__' has no attribute '_label_clip_name'

tests\test_worker.py:204: AttributeError
===== short test summary info =====
FAILED tests/test_worker.py::test_cmd_train_detector_joins_dated_labels_to_bare_videos
FAILED
tests/test_worker.py::test_label_clip_name_strips_date_prefix[2026-06-22_GX020094.rallies.csv-
GX020094]
FAILED tests/test_worker.py::test_label_clip_name_strips_date_prefix[GX020094.rallies.csv-
GX020094]
FAILED tests/test_worker.py::test_label_clip_name_strips_date_prefix[GX020094_proxy.rallies.csv-
GX020094_proxy]
FAILED tests/test_worker.py::test_label_clip_name_strips_date_prefix[2026-06-
21_mahadevpura_1.rallies.csv-mahadevpura_1]
FAILED tests/test_worker.py::test_label_clip_name_strips_date_prefix[v1.csv-v1]
FAILED tests/test_worker.py::test_label_clip_name_strips_date_prefix[2026-06-22_GX020094.csv-
GX020094]
7 failed in 2.45s
=== restore the fix ===
no changes added to commit (use "git add" and/or "git commit -a")
Dropped refs/stash@{0} (4c9bed8282fa8e721bcbe5a4243621c22b3ff302)
[/external_agent_tool_result]
```

59. assistant (2026-07-02T06:11:32.851Z)

The new tests fail without the fix (helper tests `AttributeError`; the regression test hits rc 2) and pass with it ? a genuine regression guard.

```
**Pre-LGTM gate summary:**
- ? Full suite: 1426 passed; 1 failure (`test_served_gate_a_regression`) **proven pre-existing**
on clean `origin/master` ? local golden-corpus snapshot drift, untouched by this change.
- ? Coverage 85.51% ? 70% floor (`backend/worker/__main__.py` at 88%).
- ? ruff: my files clean; the 2 errors are in untracked owner WIP (`artifacts/`), absent from
CI.
- ? mypy: clean (136 files).
```

Now commit (explicit paths only, per branching-safety) and open the PR:

60. assistant (2026-07-02T06:11:32.851Z)

```
[external_agent_tool_call: Bash]
description: Stage explicit paths and verify
command: git add backend/worker/__main__.py tests/test_worker.py
echo "=== staged (should be ONLY my 2 files) ==="; git diff --cached --name-only
echo "=== re-verify HEAD is mine ==="; git branch --show-current; git log --oneline
origin/master..HEAD | head
[/external_agent_tool_call]
```

61. assistant (2026-07-02T06:11:32.851Z)

```
[external_agent_tool_result]
=== staged (should be ONLY my 2 files) ===
backend/worker/__main__.py
tests/test_worker.py
=== re-verify HEAD is mine ===
fix/worker-train-dated-label-join
```

```
[/external_agent_tool_result]
```

62. assistant (2026-07-02T06:11:32.851Z)

Only my 2 files staged, branch cleanly based on `origin/master`. Committing:

63. assistant (2026-07-02T06:11:32.851Z)

```
[external_agent_tool_call: Bash]
```

```
description: Commit the fix
```

```
command: git commit -F - <<'EOF'
```

```
fix(worker): strip ISO-date prefix when joining labels to videos in train
```

`worker train --trajectory-source detector` derived the per-clip join key straight from the label filename (`fname.replace(".rallies.csv","")`), e.g. `2026-06-22\_GX020094`, then looked it up against the bare video stem `GX020094` in `video\_by\_stem`. Under the locked canonical naming `labels/<YYYY-MM-DD>\_<name>.rallies.csv` (docs/DATA_IN_GCS.md §7.1) the date prefix means stems never match ? every clip hits "no video ? skipping" ? train aborts with "<2 videos" (§7b blocker #1). synth-train + all eval are unaffected (they don't join videos).

Factor the label?clip-name derivation into `\_label\_clip\_name()`, which strips both the `.rallies.csv`/`.csv` suffix and an optional leading `^d{4}-d{2}-d{2}\_` prefix, so a dated label and a legacy bare label both resolve to the bare `videos/<name>` stem. The downstream spec `name` is now the clean (date-stripped) name.

Tests: parametrized helper cases (dated / bare / `\_proxy` / underscores-in-name / bare `.csv`) + a cmd_train detector regression that joins dated labels to bare-stem videos (rc 0, 2 specs) ? both fail without the fix.

Co-Authored-By: Claude Opus 4.8 <noreply@anthropic.com>

```
EOF
```

```
echo "=== committed ==="; git log --oneline -1
```

```
[/external_agent_tool_call]
```

64. assistant (2026-07-02T06:11:32.851Z)

```
[external_agent_tool_result]
```

```
[fix/worker-train-dated-label-join a2c8be0] fix(worker): strip ISO-date prefix when joining labels to videos in train
```

```
2 files changed, 86 insertions(+), 2 deletions(-)
```

```
=== committed ===
```

```
a2c8be0 fix(worker): strip ISO-date prefix when joining labels to videos in train
```

```
[/external_agent_tool_result]
```

65. assistant (2026-07-02T06:11:32.851Z)

Now push and open the PR:

66. assistant (2026-07-02T06:11:32.852Z)

```
[external_agent_tool_call: Bash]
```

```
description: Push branch to origin
```

```
command: git push -u origin fix/worker-train-dated-label-join 2>&1 | tail -8
```

```
[/external_agent_tool_call]
```

67. assistant (2026-07-02T06:11:32.852Z)

```
[external_agent_tool_result]
```

```
remote:
```

```
remote: Create a pull request for 'fix/worker-train-dated-label-join' on GitHub by visiting:
```

```
remote:      https://github.com/Khelsutra/badminton-highlight-indexer/pull/new/fix/worker-train-
```

```

dated-label-join
remote:
branch 'fix/worker-train-dated-label-join' set up to track 'origin/fix/worker-train-dated-label-join'.
To https://github.com/Khelsutra/badminton-highlight-indexer.git
 * [new branch]      fix/worker-train-dated-label-join -> fix/worker-train-dated-label-join
[/external_agent_tool_result]

```

68. assistant (2026-07-02T06:11:32.852Z)

```

[external_agent_tool_call: Bash]
description: Open the pull request
command: gh pr create --base master --head fix/worker-train-dated-label-join \
  --title "fix(worker): strip ISO-date prefix when joining labels to videos in train" \
  --body "$(cat <<'EOF'

```

Problem

`worker train --trajectory-source detector` can't match labels to videos under the **locked canonical GCS naming** ``labels/<YYYY-MM-DD>_<name>.rallies.csv`` (docs/DATA_IN_GCS.md §7.1). This is **\$7b blocker #1**.

Root cause (code-verified): ``cmd_train`` derived the per-clip key straight from the label filename `? `fname.replace(".rallies.csv","").replace(".csv","")`` `? `2026-06-22_GX020094`` (`[backend/worker/__main__.py:384](backend/worker/__main__.py)`) `? then looked up the video via `video_by_stem.get(name)``, where ``video_by_stem`` is keyed by the **bare** stem ``GX020094`` (``videos/<name>.mp4``). The date prefix means stems never match `? every clip hits the `no video ? skipping` branch` `? `worker train --trajectory-source detector --corpus-uri gcs://khelsutra-rally-corpus` aborts with `<2 videos`.`

``synth` train + all eval are unaffected` (they don't join videos).

Fix

Factor the label?clip-name derivation into a small helper ``_label_clip_name()`` that strips **both** the ``.rallies.csv``/``.csv`` suffix **and** an optional leading ``${4}-${2}-${2}_`` prefix, so a dated label and a legacy bare label both resolve to the bare ``videos/<name>`` stem. The ``${_proxy}`` suffix and underscores inside ``${_name}`` are preserved. The downstream spec ``${_name}`` is now the clean (date-stripped) name.

...

```

2026-06-22_GX020094.rallies.csv      -> GX020094      (canonical, dated)
GX020094.rallies.csv                -> GX020094      (legacy bare ? same stem)
GX020094_proxy.rallies.csv          -> GX020094_proxy (_proxy kept)
2026-06-21_mahadevpura_1.rallies.csv -> mahadevpura_1 (underscores survive)
...

```

Tests (both fail without the fix)

```

- `test_label_clip_name_strips_date_prefix` ? parametrized: dated / bare / `${_proxy}` /
underscores-in-name / bare`.csv` / dated+bare-`.csv`.
- `test_cmd_train_detector_joins_dated_labels_to_bare_videos` ? `cmd_train --trajectory-source
detector` (stub runner) with dated labels joining to bare-stem videos ? rc 0, 2 specs,
names date-stripped.

```

Verification (repo pre-LGTM gate)

```

- `python run_all_tests.py --cov=backend --cov=training --cov-fail-under=70` ? 1426 passed,
coverage 85.51% (worker module 88%). The one failure, `test_served_gate_a_regression`, is a
pre-existing local golden-corpus snapshot drift (reproduced identically on a clean
`origin/master` tree with this change reverted) ? unrelated to and untouched by this PR.
- `ruff check` on the changed files ? clean. (Repo-wide ruff flags only untracked owner WIP
under `artifacts/`, absent from CI.)
- `mypy backend training` ? clean (136 files).

```

? Generated with [Claude Code](https://claude.com/claude-code)

EOF

```
)" 2>&1 | tail -5  
[/external_agent_tool_call]
```

69. assistant (2026-07-02T06:11:32.852Z)

```
[external_agent_tool_result]  
https://github.com/Khelsutra/badminton-highlight-indexer/pull/319  
[/external_agent_tool_result]
```

70. assistant (2026-07-02T06:11:32.852Z)

Done. PR opened: **[badminton-highlight-indexer#319](https://github.com/Khelsutra/badminton-highlight-indexer/pull/319)**

What I did

Fixed ``cmd_train``'s label?video join so it works under the canonical GCS naming ``labels/<YYYY-MM-DD>_<name>.rallies.csv`` (the ``docs/DATA_IN_GCS.md`` §7b blocker #1).

****The fix**** ? factored the clip-name derivation into a tested helper in `[backend/worker/__main__.py](backend/worker/__main__.py)`:

```
```python  
_LABEL_DATE_PREFIX = re.compile(r"^\d{4}-\d{2}-\d{2}_")

def _label_clip_name(fname: str) -> str:
 name = fname.replace(".rallies.csv", "").replace(".csv", "")
 return _LABEL_DATE_PREFIX.sub("", name)
...`
```

``cmd_train`` now calls ``name = _label_clip_name(fname)``, so ``2026-06-22_GX020094.rallies.csv`` ? ``GX020094`` and joins to ``videos/GX020094.mp4``. ``_proxy`` and underscores inside ``<name>`` are preserved; legacy bare labels still map to the same stem. The downstream spec ``name`` is the clean, date-stripped name. ``import re`` added at module level.

**\*\*Tests\*\*** (both fail without the fix, pass with it):

- ``test_label_clip_name_strips_date_prefix`` ? parametrized over dated / bare / ``_proxy`` / underscored / bare-``.csv`` cases.
- ``test_cmd_train_detector_joins_dated_labels_to_bare_videos`` ? end-to-end ``--trajectory-source detector`` regression: dated labels ? bare-stem videos ? rc 0, 2 specs.

## Verification (repo pre-LGTM gate)

```
| Check | Result |
|---|---|
| `run_all_tests.py --cov=backend --cov=training --cov-fail-under=70` | **1426 passed**,
coverage **85.51%** (worker module 88%) |
| `ruff check` (changed files) | clean |
| `mypy backend training` | clean (136 files) |
```

**\*\*One honest caveat:\*\*** the full suite shows **\*\*1 failure\*\***, ``test_served_gate_a_regression::test_proposal_windowing_gate_a_is_f1_neutral_and_cuts_over_seg`` (asserts ``mean_seg_off == 1.16 ± 0.10``, got ``1.36``). I verified it's **\*\*pre-existing\*\*** ? it reproduces identically on a clean ``origin/master`` tree with my changes stashed. It's local golden-corpus snapshot drift (the in-flight ``golden-corpus-hygiene`` label cleanup changed the fixture set), on a code path this PR never touches. The repo-wide ruff errors are likewise in untracked owner WIP under ``artifacts/``, which won't exist in CI.

Branched off ``origin/master``, staged explicit paths only, one PR for this item. I left ``docs/DATA_IN_GCS.md`` §7b untouched ? that doc lives on the ``work/golden-corpus-hygiene`` branch, not master, so flipping blocker #1 to resolved (referencing PR #319) belongs with that branch's merge, not here.

## 71. user (2026-07-02T06:11:32.852Z)

can you also do a doc fix related to this and merge the PRs once the coverage has been upheld/improved and the docs fixed

## 72. assistant (2026-07-02T06:11:32.852Z)

Let me gather the facts I need before deciding where the doc fix goes and how to handle the merge ? CI status, whether the doc even exists on master, whether there's a PR for the corpus branch, and exactly what's uncovered in the worker module.

## 73. assistant (2026-07-02T06:11:32.852Z)

```
[external_agent_tool_call: Bash]
description: Investigate CI, doc location, open PRs
command: echo "=== PR #319 CI / mergeable state ==="; gh pr view 319 --json
mergeable,mergeStateStatus,statusCheckRollup,headRefName,baseRefName 2>&1 | head -60
echo; echo "=== does DATA_IN_GCS.md exist on origin/master? ==="; git cat-file -e
origin/master:docs/DATA_IN_GCS.md 2>&1 && echo "YES on master" || echo "NO ? not on master"
echo; echo "=== which branches have the doc? ==="; git branch -a --contains $(git rev-list -1
origin/work/golden-corpus-hygiene -- docs/DATA_IN_GCS.md 2>/dev/null) 2>&1 | head
echo; echo "=== open PRs ==="; gh pr list --state open --json
number,title,headRefName,baseRefName 2>&1 | head -40
[/external_agent_tool_call]
```

## 74. assistant (2026-07-02T06:11:32.852Z)

```
[external_agent_tool_result]
=== PR #319 CI / mergeable state ===
{"baseRefName":"master","headRefName":"fix/worker-train-dated-label-join","mergeStateStatus":"UN
STABLE","mergeable":"MERGEABLE","statusCheckRollup":{"__typename":"CheckRun","completedAt":"202
6-06-22T12:19:59Z","conclusion":"SUCCESS","detailsUrl":"https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/27952140582/job/82711391288","name":"Ruff lint","startedAt":"2026
-06-
22T12:19:50Z","status":"COMPLETED","workflowName":"CI"},"__typename":"CheckRun","completedAt":"
2026-06-
22T12:19:56Z","conclusion":"SUCCESS","detailsUrl":"https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/27952140582/job/82711391282","name":"Leak scan (PII / attribution
guard)","startedAt":"2026-06-
22T12:19:50Z","status":"COMPLETED","workflowName":"CI"},"__typename":"CheckRun","completedAt":"
2026-06-
22T12:20:18Z","conclusion":"SUCCESS","detailsUrl":"https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/27952140582/job/82711391389","name":"Mypy (lenient baseline)","st
artedAt":"2026-06-
22T12:19:50Z","status":"COMPLETED","workflowName":"CI"},"__typename":"CheckRun","completedAt":"
2026-06-
22T12:20:12Z","conclusion":"SUCCESS","detailsUrl":"https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/27952140582/job/82711391575","name":"Syntax sweep + import smoke
(no GPU stack)","startedAt":"2026-06-
22T12:19:50Z","status":"COMPLETED","workflowName":"CI"},"__typename":"CheckRun","completedAt":"
0001-01-01T00:00:00Z","conclusion":"","detailsUrl":"https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/27952140582/job/82711391441","name":"Full test suite (offline, mo
cked)","startedAt":"2026-06-
22T12:19:50Z","status":"IN_PROGRESS","workflowName":"CI"},"__typename":"CheckRun","completedAt"
:"2026-06-
22T12:20:14Z","conclusion":"SUCCESS","detailsUrl":"https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/27952140582/job/82711391421","name":"Golden fixtures cross-OS
(ubuntu-latest)","startedAt":"2026-06-
22T12:19:50Z","status":"COMPLETED","workflowName":"CI"},"__typename":"CheckRun","completedAt":"
0001-01-01T00:00:00Z","conclusion":"","detailsUrl":"https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/27952140582/job/82711391318","name":"Golden fixtures cross-OS
(windows-latest)","startedAt":"2026-06-
22T12:19:51Z","status":"IN_PROGRESS","workflowName":"CI"},"__typename":"CheckRun","completedAt"
:"2026-06-
```

```
22T12:20:14Z", "conclusion": "SUCCESS", "detailsUrl": "https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/27952140582/job/82711391409", "name": "Golden fixtures cross-OS (macos-latest)", "startedAt": "2026-06-22T12:19:50Z", "status": "COMPLETED", "workflowName": "CI"}, {"__typename": "CheckRun", "completedAt": "2026-06-22T12:20:21Z", "conclusion": "SUCCESS", "detailsUrl": "https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/27952140582/job/82711391301", "name": "Dependency license scan (no copyleft in runtime deps)", "startedAt": "2026-06-22T12:19:50Z", "status": "COMPLETED", "workflowName": "CI"}, {"__typename": "CheckRun", "completedAt": "2026-06-22T12:20:58Z", "conclusion": "SUCCESS", "detailsUrl": "https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/27952140582/job/82711391443", "name": "Built wheel smoke (install + launch)", "startedAt": "2026-06-22T12:19:50Z", "status": "COMPLETED", "workflowName": "CI"}]]}
```

```
=== does DATA_IN_GCS.md exist on origin/master? ===
fatal: path 'docs/DATA_IN_GCS.md' does not exist in 'origin/master'
NO ? not on master
```

```
=== which branches have the doc? ===
work/golden-corpus-hygiene
remotes/origin/work/golden-corpus-hygiene
```

```
=== open PRs ===
[{"baseRefName": "master", "headRefName": "feat/nightly-scheduler-fix", "number": 320, "title": "feat(nightly): make the Cloud Run scheduler path work E2E + turnkey email/trigger"}, {"baseRefName": "master", "headRefName": "fix/worker-train-dated-label-join", "number": 319, "title": "fix(worker): strip ISO-date prefix when joining labels to videos in train"}, {"b...
[/external_agent_tool_result]
```

## 75. assistant (2026-07-02T06:11:32.852Z)

Key findings that shape the plan:

- **PR #319 CI**: nearly all green (ruff, mypy, leak scan, license scan, wheel smoke, golden-fixtures macOS/Ubuntu all ?); "Full test suite" + "Golden fixtures windows-latest" still **in progress**. `mergeable: MERGEABLE``.
- **docs/DATA\_IN\_GCS.md** is NOT on master ? it lives only on `work/golden-corpus-hygiene``, which is **open PR #316** (<https://github.com/Khelsutra/badminton-highlight-indexer/pull/316>).
- So the doc fix belongs on #316's branch, not #319.
- "The PRs" = **#319 (code fix)** + **#316 (doc map)**, both this session's track. #320 is an unrelated nightly-scheduler item.

So: improve coverage on #319, add the resolved-blocker doc fix to #316, then merge both once CI is green. Let me get the authoritative coverage gaps for the worker module and re-read the doc tracker sections in parallel.

## 76. assistant (2026-07-02T06:11:32.852Z)

```
[external_agent_tool_call: Bash]
description: Worker-test coverage with missing lines
command: python -m pytest tests/test_worker.py -q --cov=backend.worker --cov-report=term-missing 2>&1 | tail -15
[/external_agent_tool_call]
```

## 77. assistant (2026-07-02T06:11:32.852Z)

```
[external_agent_tool_result]
Command running in background with ID: b51kt55u4. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\73a3938e-1da3-4e28-bfa7-346a15718301\tasks\b51kt55u4.output. You will be notified when it completes. To check interim output, use Read on that file path.
[/external_agent_tool_result]
```

78. assistant (2026-07-02T06:11:32.852Z)

While coverage runs, let me confirm the working-tree state of the doc (it shouldn't exist on my current branch since it's not on master) and read the exact tracker lines from the corpus branch via git:

79. assistant (2026-07-02T06:11:32.852Z)

```
[external_agent_tool_call: Bash]
description: Check doc presence and branch divergence
command: echo "=== does the doc exist in my current working tree? ==="; test -f
docs/DATA_IN_GCS.md && echo "present" || echo "absent (expected ? not on this branch)"
echo; echo "=== current branch ==="; git branch --show-current
echo; echo "=== PR #316 head sha + is its branch ff-able / behind master? ==="; git rev-list
--left-right --count origin/master...origin/work/golden-corpus-hygiene 2>&1
echo "(left=master-ahead right=branch-ahead)"
echo; echo "=== any uncommitted tracked changes right now? ==="; git status --porcelain 2>&1 |
grep -v '^??' || echo "(none ? clean tracked tree)"
[/external_agent_tool_call]
```

80. assistant (2026-07-02T06:11:32.852Z)

```
[external_agent_tool_result]
=== does the doc exist in my current working tree? ===
absent (expected ? not on this branch)

=== current branch ===
fix/worker-train-dated-label-join

=== PR #316 head sha + is its branch ff-able / behind master? ===
4 4
(left=master-ahead right=branch-ahead)

=== any uncommitted tracked changes right now? ===
(none ? clean tracked tree)
[/external_agent_tool_result]
```

81. assistant (2026-07-02T06:11:32.852Z)

Coverage output is ready. Let me read it and check PR #316's mergeable state (it's diverged ? 4 ahead / 4 behind master):

82. assistant (2026-07-02T06:11:32.852Z)

```
[external_agent_tool_call: Read]
file: C:\Users\avidu\AppData\Local\Temp\claudes-C--Users-avidu-Projects-badminton-highlight-
indexer\73a3938e-1da3-4e28-bfa7-346a15718301\tasks\b51kt55u4.output
[/external_agent_tool_call]
```

83. assistant (2026-07-02T06:11:32.852Z)

```
[external_agent_tool_result]
1 [100%]
2 ===== tests coverage =====
3 _____ coverage: platform win32, python 3.13.13-final-0 _____
4
5 Name Stmts Miss Cover Missing
6 -----
7 backend\worker__init__.py 0 0 100%
8 backend\worker__main__.py 300 55 82% 114-115, 126-139, 146-155, 162, 170,
204-206, 211-212, 260, 309-314, 333-335, 338-340, 350-351, 372, 386-388, 413-414, 422-423,
427-428, 439-440, 451-453, 457-458, 542
9 -----
10 TOTAL 300 55 82%
```

```
11 26 passed in 3.16s
12
[/external_agent_tool_result]
```

#### 84. assistant (2026-07-02T06:11:32.852Z)

82% from worker tests alone (55 missed); the full-suite figure is 88%. The most valuable, directly-related uncovered branch is the `***no video for `{name}` ? skipping***` path (the exact failure mode my fix addresses). Let me read the current ``cmd_train`` region to map the uncovered lines precisely.

#### 85. assistant (2026-07-02T06:11:32.852Z)

```
[external_agent_tool_call: Read]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\backend\worker__main__.py
[/external_agent_tool_call]
```

#### 86. assistant (2026-07-02T06:11:32.853Z)

```
[external_agent_tool_result]
400 if not vname.lower().endswith(_VIDEO_EXTS):
401 continue # skip sidecars (.srt/.json/thumbnails) so they can't shadow
the video
402 video_by_stem[os.path.splitext(vname)[0]] = vuri
403
404 print(f"[worker] trajectory source: {args.trajectory_source}")
405 specs: List[dict] = []
406 for uri in label_uris:
407 fname = storage.parse_uri(uri).name # e.g.
2026-06-22_GX020094.rallies.csv
408 name = _label_clip_name(fname) # -> GX020094 (date
prefix stripped)
409 local_label = storage.fetch(uri, os.path.join(labels_dir, fname),
config=storage_cfg)
410 if args.trajectory_source == "detector":
411 vuri = video_by_stem.get(name)
412 if not vuri:
413 print(f"[worker] no video for {name!r} under {corpus}/videos/ ?
skipping.")
414 continue
415 local_video = storage.fetch(vuri, os.path.join(videos_dir,
storage.parse_uri(vuri).name),
416 config=storage_cfg)
417 video_id = compute_video_id(local_video)
418 # Real fps/width drive the trajectory frame->time mapping; PROBE (don't
guess) ? a
419 # wrong fps silently mis-aligns every label, so skip the video if ffprobe
can't read it.
420 meta = _probe_video_fps_width(local_video)
421 if meta is None:
422 print(f"[worker] could not probe fps/width for {name!r} (ffprobe) ?
skipping (won't guess).")
423 continue
424 fps, frame_width = meta
425 traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
426 if not traj:
427 print(f"[worker] detector produced no trajectory for {name!r} ?
skipping.")
428 continue
429 specs.append({"name": name, "trajectory": traj, "labels": local_label,
430 "fps": fps, "frame_width": frame_width})
431 else:
432 from backend.pipeline.detectors.stub_runner import
synth_trajectory_from_labels
```



```
[/external_agent_tool_result]
```

### 93. assistant (2026-07-02T06:11:32.853Z)

Coverage improved 82% ? 84% (lines 412-414, 421-423, 426-428 now covered; 28 tests pass). One more cheap, thematically-perfect win: line 438-440 is the literal ``<2 videos`` about the bug tripped for `*every*` clip ? let me cover it via a legitimate single-join case:

### 94. assistant (2026-07-02T06:11:32.853Z)

```
[external_agent_tool_call: Edit]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_worker.py
[/external_agent_tool_call]
```

### 95. assistant (2026-07-02T06:11:32.853Z)

```
[external_agent_tool_result]
The file C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_worker.py has been
updated successfully. (file state is current in your context ? no need to Read it back)
[/external_agent_tool_result]
```

### 96. assistant (2026-07-02T06:11:32.853Z)

```
[external_agent_tool_call: Bash]
description: Run worker tests with coverage
command: python -m pytest tests/test_worker.py -q --cov=backend.worker --cov-report=term-missing
2>&1 | tail -11
[/external_agent_tool_call]
```

### 97. assistant (2026-07-02T06:11:32.853Z)

```
[external_agent_tool_result]
..... [100%]
===== tests coverage =====
_____ coverage: platform win32, python 3.13.13-final-0 _____

Name Stmts Miss Cover Missing

backend\worker__init__.py 0 0 100%
backend\worker__main__.py 300 47 84% 114-115, 126-139, 146-155, 162, 170, 204-206,
211-212, 260, 309-314, 333-335, 338-340, 350-351, 372, 386-388, 451-453, 457-458, 542

TOTAL 300 47 84%
29 passed in 3.23s
[/external_agent_tool_result]
```

### 98. assistant (2026-07-02T06:11:32.853Z)

Now 29 tests pass; worker coverage 47 missed (84%, up from 82%) ? the join-miss, probe-fail, no-trajectory, and ``<2 videos`` branches are all covered now. Let me commit these test additions to #319 and run the full pre-LGTM gate to confirm overall coverage holds.

### 99. assistant (2026-07-02T06:11:32.853Z)

```
[external_agent_tool_call: Bash]
description: Commit test additions to #319
command: git add tests/test_worker.py
echo "=== staged ==="; git diff --cached --name-only
echo "=== verify HEAD is mine ==="; git branch --show-current; git log --oneline
origin/master..HEAD
git commit -F - <<'EOF'
test(worker): cover cmd_train per-label join/skip branches
```

Adds coverage for the detector-source per-label loop branches around the

label?video join that this PR touches: the join-miss skip (label with no matching video), the fps/width probe-fail skip, the no-trajectory skip, and the `<2 videos` leave-one-video-out abort (the exact guard the date-prefix bug tripped for every clip). Worker-module line coverage 82% -> 84%.

Co-Authored-By: Claude Opus 4.8 <noreply@anthropic.com>

EOF

echo "=== committed ==="; git log --oneline -2

[/external\_agent\_tool\_call]

#### 100. assistant (2026-07-02T06:11:32.853Z)

[external\_agent\_tool\_result]

=== staged ===

tests/test\_worker.py

=== verify HEAD is mine ===

fix/worker-train-dated-label-join

a2c8be0 fix(worker): strip ISO-date prefix when joining labels to videos in train

[fix/worker-train-dated-label-join b3b4952] test(worker): cover cmd\_train per-label join/skip branches

1 file changed, 131 insertions(+)

=== committed ===

b3b4952 test(worker): cover cmd\_train per-label join/skip branches

a2c8be0 fix(worker): strip ISO-date prefix when joining labels to videos in train

[/external\_agent\_tool\_result]

#### 101. assistant (2026-07-02T06:11:32.854Z)

Now run the full pre-LGTM gate locally to confirm overall coverage holds and no regressions, then push:

#### 102. assistant (2026-07-02T06:11:32.854Z)

[external\_agent\_tool\_call: Bash]

description: Push and run full coverage gate

command: git push origin fix/worker-train-dated-label-join 2>&1 | tail -3

echo "=== full suite + coverage (the gate) ==="

python run\_all\_tests.py --cov=backend --cov=training --cov-fail-under=70 2>&1 | tail -8

[/external\_agent\_tool\_call]

#### 103. assistant (2026-07-02T06:11:32.854Z)

[external\_agent\_tool\_result]

To <https://github.com/Khelsutra/badminton-highlight-indexer.git>

a2c8be0..b3b4952 fix/worker-train-dated-label-join -> fix/worker-train-dated-label-join

=== full suite + coverage (the gate) ===

TOTAL 10001 1441 86%

Required test coverage of 70% reached. Total coverage: 85.59%

===== short test summary info =====

FAILED tests/test\_served\_gate\_a\_regression.py::test\_proposal\_windowing\_gate\_a\_is\_f1\_neutral\_and\_cuts\_over\_seg

1 failed, 1429 passed, 7 warnings in 147.40s (0:02:27)

=====

RESULT: FAIL - pytest exit 1

=====

[/external\_agent\_tool\_result]

#### 104. assistant (2026-07-02T06:11:32.854Z)

Full suite: **\*\*1429 passed\*\***, overall coverage **\*\*85.59%\*\*** (up from 85.51%), only the same pre-existing `test\_served\_gate\_a\_regression` failure. Critical question: does that test run/fail in CI too? It depends on local golden-corpus presence. Let me check PR #319's actual CI result and the test's skip condition.

#### 105. assistant (2026-07-02T06:11:32.854Z)

```
[external_agent_tool_call: Bash]
description: Check CI result and test skip guard
command: echo "=== PR #319 full-suite CI job result ==="; gh pr view 319 --json
statusCheckRollup --jq '.statusCheckRollup[] | select(.name=="Full test suite (offline,
mocked)") | {name, status, conclusion}' 2>&1
echo; echo "=== overall mergeStateStatus ==="; gh pr view 319 --json mergeable,mergeStateStatus
--jq '{mergeable,mergeStateStatus}' 2>&1
echo; echo "=== how does test_served_gate_a_regression source its corpus? (skip guard?) ===";
sed -n '1,60p' tests/test_served_gate_a_regression.py 2>/dev/null | grep -nE
"skip|getenv|environ|isfile|exists|glob|manifest|GOLDEN|corpus|def test_" | head -40
[/external_agent_tool_call]
```

#### 106. assistant (2026-07-02T06:11:32.854Z)

```
[external_agent_tool_result]
=== PR #319 full-suite CI job result ===
{"conclusion":"SUCCESS","name":"Full test suite (offline, mocked)","status":"COMPLETED"}

=== overall mergeStateStatus ===
{"mergeStateStatus":"UNSTABLE","mergeable":"MERGEABLE"}

=== how does test_served_gate_a_regression source its corpus? (skip guard?) ===
21:On that corpus the served path is **recall-starved**: WASB tracks a single shuttle while
several
24:windows, on this under-segmented corpus it is mildly **F1-negative** (mean served ?F1 ?
-0.034),
28:The canonical **+0.074 / 6-of-6 / p=0.031** Gate-A win was a property of the *original-6*
corpus
31:**present-corpus** behaviour so a future windowing/corpus change that silently shifts it
trips the
33:re-measurement as the corpus grows.
43:# (CI), skip the whole module rather than erroring at import.
44:pytest.importorskip("cv2")
53:# The golden manifest lives in the MAIN checkout's output/ (gitignored; absent in a fresh
57:MANIFEST = os.environ.get(
59: os.path.join(_REPO_ROOT, "output", "human_lovo_manifest.json"),
[/external_agent_tool_result]
```

#### 107. assistant (2026-07-02T06:11:32.854Z)

**\*\*Confirmed\*\***: PR #319's "Full test suite" CI job = **\*\*SUCCESS\*\***. The  
`test\_served\_gate\_a\_regression` failure is local-only ? the test sources  
`output/human\_lovo\_manifest.json` (gitignored, **\*\*absent in CI\*\*** ? skipped), so CI never sees the  
local corpus drift. This fully validates that the local failure is environmental, not a real  
regression.

Let me confirm the complete CI rollup has zero failures (and why it shows UNSTABLE):

#### 108. assistant (2026-07-02T06:11:32.854Z)

```
[external_agent_tool_call: Bash]
description: Full CI rollup for #319
command: echo "=== ALL checks for #319 (name | status | conclusion) ==="; gh pr view 319 --json
statusCheckRollup --jq '.statusCheckRollup[] | "\(.conclusion // .status)\t\(.name)"' 2>&1 |
sort | uniq -c
echo; echo "=== any non-SUCCESS? ==="; gh pr view 319 --json statusCheckRollup --jq
'.statusCheckRollup[] | select((.conclusion // "PENDING") != "SUCCESS") | "\(.conclusion //
.status)\t\(.name)"' 2>&1 || echo "(all success)"
[/external_agent_tool_call]
```

#### 109. assistant (2026-07-02T06:11:32.854Z)

```
[external_agent_tool_result]
=== ALL checks for #319 (name | status | conclusion) ===
1 FAILURE Ruff lint
1 SUCCESS Built wheel smoke (install + launch)
1 SUCCESS Dependency license scan (no copyleft in runtime deps)
1 SUCCESS Full test suite (offline, mocked)
1 SUCCESS Golden fixtures cross-OS (macos-latest)
1 SUCCESS Golden fixtures cross-OS (ubuntu-latest)
1 SUCCESS Golden fixtures cross-OS (windows-latest)
1 SUCCESS Leak scan (PII / attribution guard)
1 SUCCESS Mypy (lenient baseline)
1 SUCCESS Syntax sweep + import smoke (no GPU stack)

=== any non-SUCCESS? ===
FAILURE Ruff lint
[/external_agent_tool_result]
```

#### 110. assistant (2026-07-02T06:11:32.854Z)

CI now flags **\*\*Ruff lint = FAILURE\*\*** on my latest push ? my new test code must have a violation (CI only sees committed files; `artifacts/` is untracked). Let me reproduce exactly:

#### 111. assistant (2026-07-02T06:11:32.854Z)

```
[external_agent_tool_call: Bash]
description: Reproduce ruff failure on test file
command: echo "=== ruff on my test file ==="; ruff check tests/test_worker.py 2>&1 | tail -25
echo; echo "=== ruff config the CI uses? ==="; grep -rn "ruff" .github/workflows/*.yaml
2>/dev/null | head; echo "---"; sed -n '1,40p' pyproject.toml 2>/dev/null | grep -nA20 "ruff" |
head -30
[/external_agent_tool_call]
```

#### 112. assistant (2026-07-02T06:11:32.854Z)

```
[external_agent_tool_result]
=== ruff on my test file ===
F401 [*] `os` imported but unused
--> tests\test_worker.py:359:12
 |
357 | This is the guard the date-prefix bug used to trip for EVERY clip (DATA_IN_GCS $7b
#1); here it
358 | fires legitimately because only one of two labels has a matching video.""
359 | import os
 | ^^
360 | import subprocess
 |
help: Remove unused import: `os`

Found 1 error.
[*] 1 fixable with the `--fix` option.
```

```
=== ruff config the CI uses? ===
16: # Lint gate (ruff.toml at repo root): pyflakes/pycodestyle + bugbear.
17: # Also the repo-wide syntax guard ? ruff reports invalid-syntax on every
28: - name: Install ruff
29: run: python -m pip install ruff
31: run: ruff check . --output-format github
102: # (pytest, pytest-cov, httpx for the fastapi TestClient, ruff, mypy).

[/external_agent_tool_result]
```

#### 113. assistant (2026-07-02T06:11:32.854Z)

Trivial: unused `import os` in my 4th test (it uses `subprocess` but not `os`). Let me remove

it:

**114. assistant (2026-07-02T06:11:32.854Z)**

```
[external_agent_tool_call: Edit]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_worker.py
[/external_agent_tool_call]
```

**115. assistant (2026-07-02T06:11:32.854Z)**

```
[external_agent_tool_result]
The file C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_worker.py has been
updated successfully. (file state is current in your context ? no need to Read it back)
[/external_agent_tool_result]
```

**116. assistant (2026-07-02T06:11:32.854Z)**

```
[external_agent_tool_call: Bash]
description: Verify ruff clean and tests pass
command: echo "=== ruff (whole repo minus untracked artifacts) on tracked files ==="; ruff check
tests/test_worker.py backend/worker/__main__.py 2>&1 | tail -5
echo "=== re-run worker tests to confirm still green after removing import ==="; python -m
pytest tests/test_worker.py -q 2>&1 | tail -3
[/external_agent_tool_call]
```

**117. assistant (2026-07-02T06:11:32.854Z)**

```
[external_agent_tool_result]
=== ruff (whole repo minus untracked artifacts) on tracked files ===
All checks passed!
=== re-run worker tests to confirm still green after removing import ===
..... [100%]
29 passed in 2.10s
[/external_agent_tool_result]
```

**118. assistant (2026-07-02T06:11:32.855Z)**

Ruff clean, 29 tests pass. Since this PR will squash-merge (per the repo's merge style), I'll add a small follow-up commit rather than force-push:

**119. assistant (2026-07-02T06:11:32.855Z)**

```
[external_agent_tool_call: Bash]
description: Commit ruff fix and push
command: git add tests/test_worker.py
git commit -m "$(cat <<'EOF'
style(worker-test): drop unused os import (ruff F401)

Co-Authored-By: Claude Opus 4.8 <noreply@anthropic.com>
EOF
)" 2>&1 | tail -2
git push origin fix/worker-train-dated-label-join 2>&1 | tail -2
[/external_agent_tool_call]
```

**120. assistant (2026-07-02T06:11:32.855Z)**

```
[external_agent_tool_result]
[fix/worker-train-dated-label-join 56c1415] style(worker-test): drop unused os import (ruff
F401)
 1 file changed, 1 deletion(-)
To https://github.com/Khelsutra/badminton-highlight-indexer.git
 b3b4952..56c1415 fix/worker-train-dated-label-join -> fix/worker-train-dated-label-join
[/external_agent_tool_result]
```

### 121. assistant (2026-07-02T06:11:32.855Z)

PR #319 updated; CI will re-run green. Now the doc fix on PR #316's branch. Let me switch to `work/golden-corpus-hygiene` carefully (shared-checkout safety): fetch, verify local matches origin, `--ff-only` pull.

### 122. assistant (2026-07-02T06:11:32.855Z)

```
[external_agent_tool_call: mcp_ccd_session__mark_chapter]
input: {"title":"Doc fix on corpus branch","summary":"Mark DATA_IN_GCS §7b blocker #1 resolved by PR #319 on the golden-corpus-hygiene branch (#316)"}
[/external_agent_tool_call]
```

### 123. assistant (2026-07-02T06:11:32.855Z)

```
[external_agent_tool_result]
Chapter marked: "Doc fix on corpus branch". Continue your current work.
[/external_agent_tool_result]
```

### 124. assistant (2026-07-02T06:11:32.855Z)

```
[external_agent_tool_call: Bash]
description: Switch to corpus branch safely
command: git fetch origin 2>&1 | tail -2
echo "=== local vs origin for golden-corpus-hygiene (should be 0 0) ==="; git rev-list --left-right --count work/golden-corpus-hygiene...origin/work/golden-corpus-hygiene 2>&1
echo "=== switch ==="; git checkout work/golden-corpus-hygiene 2>&1 | tail -3
echo "=== ff-only pull ==="; git pull --ff-only 2>&1 | tail -3
echo "=== confirm doc present + HEAD ==="; test -f docs/DATA_IN_GCS.md && echo "doc present";
git log --oneline -1
[/external_agent_tool_call]
```

### 125. assistant (2026-07-02T06:11:32.855Z)

```
[external_agent_tool_result]
=== local vs origin for golden-corpus-hygiene (should be 0 0) ===
fatal: ambiguous argument 'work/golden-corpus-hygiene...origin/work/golden-corpus-hygiene':
unknown revision or path not in the working tree.
Use '--' to separate paths from revisions, like this:
'git <command> [<revision>...] -- [<file>...]'
=== switch ===
error: pathspec 'work/golden-corpus-hygiene' did not match any file(s) known to git
=== ff-only pull ===
Already up to date.
=== confirm doc present + HEAD ===
56c1415 style(worker-test): drop unused os import (ruff F401)
[/external_agent_tool_result]
```

### 126. assistant (2026-07-02T06:11:32.855Z)

The local branch isn't matching ? let me list branches to see the actual state (shared checkout may have shifted):

### 127. assistant (2026-07-02T06:11:32.855Z)

```
[external_agent_tool_call: Bash]
description: List branches
command: echo "=== local branches ==="; git branch 2>&1
echo "=== remote golden-corpus-hygiene exists? ==="; git branch -r 2>&1 | grep golden
echo "=== origin/work/golden-corpus-hygiene HEAD ==="; git log --oneline -1 origin/work/golden-corpus-hygiene 2>&1
[/external_agent_tool_call]
```

### 128. assistant (2026-07-02T06:11:32.855Z)

[external\_agent\_tool\_result: error]

Exit code 128

=== local branches ===

- + bhi-execpolicy
  - chore/consolidate-fusion-schema-predicate
  - docs/adr-012-gpu-cloudrun
  - docs/archive-decoupled-compute
  - docs/archive-decoupled-v2
  - docs/compute-decoupled-serving
  - docs/court-floor-rally-end
  - docs/decouple-scope-defer
  - docs/deploy-report-polish
  - docs/desktop-app-plan
  - docs/doc-status-register
  - docs/drop-digitalocean-gcp-only
  - docs/findings-1-2-pervideo-nextsteps
  - docs/gcp-deploy-report
  - docs/gcp-pivot-runbook
  - docs/lock-m0-decisions
- + docs/lovo-resweep-n13
  - docs/m0-approved-d13-d15
  - docs/m2-finalized-report
  - docs/packaging-plan-tracked
  - docs/pkg-p2c-done
  - docs/point-i18n-ui-to-khelsutra
- + docs/rally-gap-closure
- + docs/recording-single-match
  - docs/restructure-field-roora-data
  - docs/serving-design-picks-p3-reconcile
- feat/api-m1-health-downloadurl
- feat/gcp-ops-agent-and-dryrun-guide
- feat/gdrive-storage-backend
- feat/highlights-runner
- feat/local-no-ai-segmenter
- feat/locale-analytics
- feat/localized-watermark
- feat/long-rally-lens
- feat/native-wasb-runner
- feat/packaging-p0a-app-home
- feat/packaging-p0b-config-redact
- feat/packaging-p0c-bind-host
- feat/packaging-p0d-scipy-ci
- feat/packaging-p0e-single-instance
- feat/packaging-p1a-endpoints
- feat/packaging-p1b-bundle-ui
- feat/packaging-p2a-ffmpeg
- feat/packaging-p2b-launcher
- feat/packaging-p2d-db-safety
- feat/packaging-p2e-lazy-torch
- feat/pkg-bundle-p2c-wizard
- feat/pkg-p2b-install
- feat/prominent-court-gate
- feat/serving-m1-deployment-profile
- feat/serving-m2-input-backend
- feat/serving-m3-output-base
- feat/serving-m4-device-context
- feat/serving-m5-truly-local-checks
- feat/serving-m6-cloudrun-dispatch
- feat/serving-m8-cost-ledger
- + feat/w1-hard-cap-fallback
- + feat/w2-never-empty-safeguard
- fix/compile-resolve-bucket-source
- fix/compiler-dedup-overlapping-segments
- fix/dryrun-guide-font-and-ui-tracking
- fix/engine-double-module-job-hang

```

fix/finding-b-loud-unrunnable-config
fix/fusion-loaders-mixed-schema
fix/gcp-teardown-default-delete
fix/gemini-loud-chunk-failures
fix/gemini-per-video-chunk-dir
fix/infer-observability
+ fix/runner-stitch-watermark
fix/served-gate-a-corpus-regrounding
fix/wasb-test-mocks-readability-preflight
* fix/worker-train-dated-label-join
fix/wsl-unreadable-source-preflight
+ master
ops/sync-master-d749fb9
+ perf/wasb-fast-frames
+ run/boxhill17-highlights
serving/exposure-safety-preflight
worktree-agent-a771ff1d85e18f2ed
worktree-wf_1d39e81c-851-1
worktree-wf_1d39e81c-851-3
worktree-wf_1d39e81c-851-4
worktree-wf_1d39e81c-851-5
worktree-wf_1d39e81c-851-6
worktree-wf_1d39e81c-851-7
worktree-wf_87c49086-be4-1
worktree-wf_87c49086-be4-2
worktree-wf_87c49086-be4-3
worktree-wf_87c49086-be4-4
worktree-wf_87c49086-be4-5
worktree-wf_87c49086-be4-6
worktree-wf_87c49086-be4-7
=== remote golden-corpus-hygiene exists? ===
=== origin/work/golden-corpus-hygiene HEAD ===
fatal: ambiguous argument 'origin/work/golden-corpus-hygiene': unknown revision or path not in
the working tree.
Use '--' to separate paths from revisions, like this:
'git <command> [<revision>...] -- [<file>...]'
[/external_agent_tool_result]

```

#### 129. assistant (2026-07-02T06:11:32.855Z)

The `work/golden-corpus-hygiene` branch has **\*\*vanished from local and remote\*\*** ? a sibling session or the owner likely merged/closed PR #316 in the last few minutes (classic shared-checkout shift). Let me check #316's state and whether the doc is now on master:

#### 130. assistant (2026-07-02T06:11:32.855Z)

```

[external_agent_tool_call: Bash]
description: Check if PR316 merged and doc on master
command: git fetch origin --prune 2>&1 | tail -3
echo "=== PR #316 state ==="; gh pr view 316 --json state,mergedAt,mergeCommit,headRefName --jq
'{state, mergedAt, headRefName, mergeCommit: .mergeCommit.oid}' 2>&1
echo "=== is DATA_IN_GCS.md on origin/master NOW? ==="; git cat-file -e
origin/master:docs/DATA_IN_GCS.md 2>&1 && echo "YES ? now on master" || echo "NO"
echo "=== origin/master HEAD now ==="; git log --oneline -3 origin/master 2>&1
[/external_agent_tool_call]

```

#### 131. assistant (2026-07-02T06:11:32.855Z)

```

[external_agent_tool_result]
=== PR #316 state ===
{"headRefName":"work/golden-corpus-hygiene","mergeCommit":"e221cb928ca1595102fbf7ad1bc0e64727bac
b35","mergedAt":"2026-06-22T12:29:47Z","state":"MERGED"}
=== is DATA_IN_GCS.md on origin/master NOW? ===
YES ? now on master

```

```
=== origin/master HEAD now ===
48f9d20 feat(nightly): make the Cloud Run scheduler path work E2E + turnkey email/trigger (#320)
e221cb9 docs(data): GCS corpus source-of-truth map + Phase-1 metadata migration (#316)
d301a7f fix(nightly): GCP-L4-validated fixes ? gs:// fetch dst, WASB env, weights path, pure-
engine (#318)
[/external_agent_tool_result]
```

### 132. assistant (2026-07-02T06:11:32.855Z)

Now it's clear: \*\*PR #316 merged\*\* (12:29Z, commit e221cb9) and \*\*PR #320 merged\*\* too ? so `docs/DATA\_IN\_GCS.md` is \*\*now on master\*\*, and master advanced d301a7f ? 48f9d20. This actually simplifies things: the doc fix becomes its own PR off the updated master (matching the repo's code-vs-docs PR separation, and the user's "merge the PRs" plural). Let me verify #319 is still mergeable against new master, then branch for the doc fix.

### 133. assistant (2026-07-02T06:11:32.855Z)

```
[external_agent_tool_call: Bash]
description: Verify 319 mergeable; create doc branch
command: echo "=== #319 still mergeable vs new master? ==="; gh pr view 319 --json
mergeable,mergeStateStatus --jq '{mergeable,mergeStateStatus}' 2>&1
echo "=== create doc branch off origin/master (has the doc now) ==="; git checkout -b docs/data-
gcs-train-blocker-resolved origin/master 2>&1 | tail -2
echo "=== verify HEAD + doc present ==="; git branch --show-current; git log --oneline -1; test
-f docs/DATA_IN_GCS.md && echo "doc present"
[/external_agent_tool_call]
```

### 134. assistant (2026-07-02T06:11:32.855Z)

```
[external_agent_tool_result]
=== #319 still mergeable vs new master? ===
{"mergeStateStatus":"UNSTABLE","mergeable":"MERGEABLE"}
=== create doc branch off origin/master (has the doc now) ===
branch 'docs/data-gcs-train-blocker-resolved' set up to track 'origin/master'.
Switched to a new branch 'docs/data-gcs-train-blocker-resolved'
=== verify HEAD + doc present ===
docs/data-gcs-train-blocker-resolved
48f9d20 feat(nightly): make the Cloud Run scheduler path work E2E + turnkey email/trigger (#320)
doc present
[/external_agent_tool_result]
```

### 135. assistant (2026-07-02T06:11:32.855Z)

I'm on a clean doc branch off the updated master, doc present. Let me read the exact current \$7b and \$8 text to craft a precise, honest edit:

### 136. assistant (2026-07-02T06:11:32.855Z)

```
[external_agent_tool_call: Read]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\docs\DATA_IN_GCS.md
[/external_agent_tool_call]
```

### 137. assistant (2026-07-02T06:11:32.855Z)

```
[external_agent_tool_result]
202 ---
203
204 ## 7b. Known blockers ? must-fix before the DoD (2026-06-22 adversarial review)
205
206 1. **Dated labels break `worker train --trajectory-source detector` (code-verified).**
`cmd_train`
207 derives the join key as `fname.replace(".rallies.csv","")` ? `2026-06-22_GX020094`,
then looks up
208 `video_by_stem.get(name)` against bare video stems (`GX020094`) ? so every clip is
```

```

skipped and train
209 aborts with `<2 videos`
([backend/worker/`__main__.py:384](../backend/worker/`__main__.py):384 vs
210 :378/:387). **Fix (recommended):** strip a leading `^\\d{4}-\\d{2}-\\d{2}_` from the
label-derived
211 `name` in `cmd_train` (a ~1-line change, own PR + test) ? preserves the locked human-
readable naming
212 AND unblocks detector-train. *(`synth` train + all eval are unaffected ? they don't
join videos.)*
213 2. **Phase-3 eval-on-GCS is net-new plumbing, not a flag** ? see §6 Phase-3. Eval is
local-FS only.
214 3. **`manifests/golden.json` is consumed by zero code today** (train ignores it; eval
can't fetch
215 `gs://`) ? it's correct-but-dormant until blocker #2 lands. Add a tiny manifest-
validator + a CI job
216 that authenticates to GCS and runs the litmus against it, or the DoD ("CI reads only
from `gs://`")
217 stays unverifiable.
218 4. **Gating infra (owner-only, not done):** GPU `GPUS_ALL_REGIONS` quota is still **0**
(no VM ever
219 created ? `deploy/gcp/README` §2); cloud-box secrets (SA JSON + rotated
`GEMINI_API_KEY` + WASB
220 weights staged from `models/`) aren't set up persistently. No real cloud train/infer
runs until these.
221 5. **Reproducibility:** the migration + copy helpers live in gitignored `scratch/`
222 (`migrate_golden_to_gcs.py`, `copy_videos_to_gcs.ps1`) ? promote to `scripts/` so
Phase-2/re-syncs
223 are repeatable from the repo, and add a referenced **proxy-generation** step for the
8 large F: originals.
224
225 ## 8. Progress tracker
226
227 | Item | Status |
228 |---|---|
229 | Bucket + SA + provisioning (deploy/gcp) | ? built |
230 | Cloud worker `infer/` `train` by URI | ? built (`backend/worker`) |
231 | This data-map doc | ? 2026-06-22 |
232 | Phase 1: trajectories ? `trajectories/` (15) | ? 2026-06-22 |
233 | Phase 1: labels reconciled ? `labels/<date>_<name>` (15) | ? 2026-06-22 |
234 | Phase 1: eval manifest ? `manifests/golden.json` | ? 2026-06-22 |
235 | Phase 1: remove 7 stale `_proxy/bare` label objects | ? 2026-06-22 (MD5-verified
dupes, deleted) |
236 | Phase 2: corpus videos/proxies ? `videos/<name>.mp4` | ? mapping + script done;
1/15 demo'd (`mahadevpura_1`); 14 remain ? run `rcclone` on a cloud box (preferred) or the
owner-box script |
237 | Phase 2: proxy the 8 large F: full-MP4 clips before upload | ? |
238 | Phase 3: eval/litmus default to the GCS manifest | ? |
239
240 **Definition of done:** a fresh GPU box (or CI) can run inference, the eval suite, and
Gen-0 training
241 reading **only** from `gs://khelsutra-rally-corpus` ? no `C:` / `F:` / local-Drive path
required.
242
243 ---
[/external_agent_tool_result]

```

### 138. assistant (2026-07-02T06:11:32.856Z)

Now I'll make the honest doc edit ? flip §7b blocker #1 to RESOLVED (keeping the lineage/history) with an accurate description of what landed (`\_label\_clip\_name` helper + tests), referencing PR #319, and add a tracker row in §8.

### 139. assistant (2026-07-02T06:11:32.856Z)

[external\_agent\_tool\_call: Edit]

file: C:\Users\avidu\Projects\badminton-highlight-indexer\docs\DATA\_IN\_GCS.md  
[/external\_agent\_tool\_call]

**140. assistant (2026-07-02T06:11:32.856Z)**

[external\_agent\_tool\_result]  
The file C:\Users\avidu\Projects\badminton-highlight-indexer\docs\DATA\_IN\_GCS.md has been updated successfully. (file state is current in your context ? no need to Read it back)  
[/external\_agent\_tool\_result]

**141. assistant (2026-07-02T06:11:32.856Z)**

[external\_agent\_tool\_call: Edit]  
file: C:\Users\avidu\Projects\badminton-highlight-indexer\docs\DATA\_IN\_GCS.md  
[/external\_agent\_tool\_call]

**142. assistant (2026-07-02T06:11:32.856Z)**

[external\_agent\_tool\_result]  
The file C:\Users\avidu\Projects\badminton-highlight-indexer\docs\DATA\_IN\_GCS.md has been updated successfully. (file state is current in your context ? no need to Read it back)  
[/external\_agent\_tool\_result]

**143. assistant (2026-07-02T06:11:32.856Z)**

Let me verify the doc change is clean, stage only the doc, and commit:

**144. assistant (2026-07-02T06:11:32.856Z)**

[external\_agent\_tool\_call: Bash]  
description: Verify only doc modified  
command: echo "=== status (only doc should be modified) ==="; git status --porcelain 2>&1 | grep -v '^??'  
echo "=== diff stat ==="; git diff --stat docs/DATA\_IN\_GCS.md  
echo "=== verify HEAD/base ==="; git branch --show-current; git log --oneline origin/master..HEAD | head  
[/external\_agent\_tool\_call]

**145. assistant (2026-07-02T06:11:32.856Z)**

[external\_agent\_tool\_result]  
=== status (only doc should be modified) ===  
M docs/DATA\_IN\_GCS.md  
=== diff stat ===  
docs/DATA\_IN\_GCS.md | 18 ++++++++-----  
1 file changed, 11 insertions(+), 7 deletions(-)  
=== verify HEAD/base ===  
docs/data-gcs-train-blocker-resolved  
[/external\_agent\_tool\_result]

**146. assistant (2026-07-02T06:11:32.856Z)**

Only the doc is modified, clean base. Commit, push, and open the doc PR:

**147. assistant (2026-07-02T06:11:32.856Z)**

[external\_agent\_tool\_call: Bash]  
description: Commit and push doc branch  
command: git add docs/DATA\_IN\_GCS.md  
git commit -F - <<'EOF'  
docs(data): mark DATA\_IN\_GCS \$7b blocker #1 resolved by #319  
  
The dated-label?video join bug in `worker train --trajectory-source detector`

is fixed in #319 (the `\_label\_clip\_name` helper strips the leading ISO-date prefix so `labels/<YYYY-MM-DD>\_<name>.rallies.csv` joins to `videos/<name>.mp4`). Flip \$7b blocker #1 to ? RESOLVED (keeping the lineage/history) and add a \$8 progress-tracker row, both linking the PR.

Co-Authored-By: Claude Opus 4.8 <noreply@anthropic.com>

EOF

```
git push -u origin docs/data-gcs-train-blocker-resolved 2>&1 | tail -4
[/external_agent_tool_call]
```

#### 148. assistant (2026-07-02T06:11:32.856Z)

```
[external_agent_tool_result]
[docs/data-gcs-train-blocker-resolved 62bcd4] docs(data): mark DATA_IN_GCS $7b blocker #1
resolved by #319
 1 file changed, 11 insertions(+), 7 deletions(-)
remote:
branch 'docs/data-gcs-train-blocker-resolved' set up to track 'origin/docs/data-gcs-train-
blocker-resolved'.
To https://github.com/Khelsutra/badminton-highlight-indexer.git
 * [new branch] docs/data-gcs-train-blocker-resolved -> docs/data-gcs-train-blocker-
resolved
[/external_agent_tool_result]
```

#### 149. assistant (2026-07-02T06:11:32.856Z)

```
[external_agent_tool_call: Bash]
description: Open the doc PR
command: gh pr create --base master --head docs/data-gcs-train-blocker-resolved \
 --title "docs(data): mark DATA_IN_GCS $7b blocker #1 resolved by #319" \
 --body "$(cat <<'EOF'
Companion doc update to **[#319](https://github.com/Khelsutra/badminton-highlight-
indexer/pull/319)** (the `worker train` dated-label join fix).
```

```
`docs/DATA_IN_GCS.md` $7b blocker #1 flagged that canonical labels `labels/<YYYY-MM-
DD>_<name>.rallies.csv` couldn't be matched to `videos/<name>.mp4` by `worker train
--trajectory-source detector` (every clip skipped ? `<2 videos` abort). #319 fixes it via the
`_label_clip_name()` helper, which strips both the `.rallies.csv`/`.csv` suffix and a leading
`^\\d{4}-\\d{2}-\\d{2}_` prefix.
```

This PR:

- Flips \*\*\$7b blocker #1\*\* to ? \*\*RESOLVED 2026-06-22\*\*, keeping the lineage (the problem description stays, in past tense) and pointing at the new helper location + the test coverage that landed.
- Adds a \*\*\$8 progress-tracker\*\* row recording the fix, linking #319.

Numbering of blockers #2-#5 is left unchanged (other docs/memory reference "blocker #1" by number). Docs-only change ? no code, no tests affected.

? Generated with [Claude Code](https://claude.com/claude-code)

EOF

```
)" 2>&1 | tail -3
[/external_agent_tool_call]
```

#### 150. assistant (2026-07-02T06:11:32.856Z)

```
[external_agent_tool_result]
https://github.com/Khelsutra/badminton-highlight-indexer/pull/321
[/external_agent_tool_result]
```

#### 151. assistant (2026-07-02T06:11:32.856Z)

Doc PR opened: \*\*[#321](https://github.com/Khelsutra/badminton-highlight-indexer/pull/321)\*\*.

Before merging (the consequential step), I'll adversarially verify the whole change set ? code correctness, doc-claim accuracy (no fabricated provenance), and test rigor ? via a focused verification workflow. The agents read from explicit git refs / PR diffs so they're branch-independent.

**152. assistant (2026-07-02T06:11:32.856Z)**

```
[external_agent_tool_call: Workflow]
description: Adversarial verification of fix + doc
[/external_agent_tool_call]
```

**153. assistant (2026-07-02T06:11:32.856Z)**

```
[external_agent_tool_result]
Workflow launched in background. Task ID: wt0e0iihs
Summary: Adversarially verify PR #319 (worker train dated-label join fix + tests) and PR #321 (doc resolution) before merge
Transcript dir: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\73a3938e-1da3-4e28-bfa7-346a15718301\subagents\workflows\wf_753ff171-b1d
Script file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\73a3938e-1da3-4e28-bfa7-346a15718301\workflows\scripts\verify-worker-train-fix-and-doc-wf_753ff171-b1d.js
(Edit this file with Write/Edit and re-invoke Workflow with {scriptPath: "C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\73a3938e-1da3-4e28-bfa7-346a15718301\workflows\scripts\verify-worker-train-fix-and-doc-wf_753ff171-b1d.js"} to iterate without resending the script.)
Run ID: wf_753ff171-b1d
To resume after editing the script: Workflow({scriptPath: "C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\73a3938e-1da3-4e28-bfa7-346a15718301\workflows\scripts\verify-worker-train-fix-and-doc-wf_753ff171-b1d.js", resumeFromRunId: "wf_753ff171-b1d"}) ? completed agents return cached results.

You will be notified when it completes. Use /workflows to watch live progress.
[/external_agent_tool_result]
```

**154. assistant (2026-07-02T06:11:32.856Z)**

While the verification workflow runs, let me check CI status on both PRs (the ruff fix should have turned #319 green):

**155. assistant (2026-07-02T06:11:32.856Z)**

```
[external_agent_tool_call: Bash]
description: Check CI on both PRs
command: echo "=== #319 checks ==="; gh pr view 319 --json mergeable,mergeStateStatus,statusCheckRollup --jq '{mergeable, mergeStateStatus, checks: [.statusCheckRollup[] | "\(.conclusion // .status) \(.name)"]}' 2>&1
echo; echo "=== #321 checks ==="; gh pr view 321 --json mergeable,mergeStateStatus,statusCheckRollup --jq '{mergeable, mergeStateStatus, checks: [.statusCheckRollup[] | "\(.conclusion // .status) \(.name)"]}' 2>&1
[/external_agent_tool_call]
```

**156. assistant (2026-07-02T06:11:32.856Z)**

```
[external_agent_tool_result]
=== #319 checks ===
{"checks":["SUCCESS Ruff lint","SUCCESS Leak scan (PII / attribution guard)","SUCCESS Mypy (lenient baseline)","SUCCESS Syntax sweep + import smoke (no GPU stack)","SUCCESS Full test suite (offline, mocked)","SUCCESS Golden fixtures cross-OS (ubuntu-latest)","SUCCESS Golden fixtures cross-OS (windows-latest)","SUCCESS Golden fixtures cross-OS (macos-latest)","SUCCESS Dependency license scan (no copyleft in runtime deps)","SUCCESS Built wheel smoke (install + launch)"],"mergeStateStatus":"CLEAN","mergeable":"MERGEABLE"}
```

```
=== #321 checks ===
{"checks":["SUCCESS Ruff lint","SUCCESS Leak scan (PII / attribution guard)","SUCCESS Mypy (lenient baseline)","SUCCESS Syntax sweep + import smoke (no GPU stack)"," Full test suite (offline, mocked)","SUCCESS Golden fixtures cross-OS (ubuntu-latest)","SUCCESS Golden fixtures cross-OS (windows-latest)","SUCCESS Golden fixtures cross-OS (macos-latest)","SUCCESS Dependency license scan (no copyleft in runtime deps)"," Built wheel smoke (install + launch)"],"mergeStateStatus":"UNSTABLE","mergeable":"MERGEABLE"}
[/external_agent_tool_result]
```

#### 157. assistant (2026-07-02T06:11:32.856Z)

Status before I merge:

```
- **PR [#319](https://github.com/Khelsutra/badminton-highlight-indexer/pull/319) (code fix + coverage)** ? `mergeStateStatus: CLEAN`, **all 10 CI checks green** (incl. full test suite, ruff, mypy, cross-OS golden fixtures, wheel smoke). Ready to merge.
- **PR [#321](https://github.com/Khelsutra/badminton-highlight-indexer/pull/321) (doc resolution)** ? green except "Full test suite" + "wheel smoke" still finishing (doc-only; will pass).
```

I've launched an adversarial verification workflow over the whole change set (code correctness, doc-claim accuracy / no fabricated provenance, test rigor) as the pre-merge gate. I'll merge **#319** first, then **#321** once that completes clean and **#321**'s CI finishes. Awaiting the workflow result.

#### 158. user (2026-07-02T06:11:32.856Z)

once the PRs are merged can you identify why the master/origin is broken and try to fix it?

#### 159. assistant (2026-07-02T06:11:32.857Z)

Let me check the verification workflow result and current CI state on both PRs.

#### 160. assistant (2026-07-02T06:11:32.857Z)

```
[external_agent_tool_call: Bash]
description: Check PR states
command: echo "=== #319 ==="; gh pr view 319 --json state,mergeable,mergeStateStatus --jq '{state,mergeable,mergeStateStatus}' 2>&1
echo "=== #321 ==="; gh pr view 321 --json state,mergeable,mergeStateStatus,statusCheckRollup --jq '{state,mergeable,mergeStateStatus, pending: [.statusCheckRollup[] | select((.conclusion // "PENDING")!="SUCCESS") | .name]}' 2>&1
[/external_agent_tool_call]
```

#### 161. assistant (2026-07-02T06:11:32.857Z)

```
[external_agent_tool_result]
=== #319 ===
{"mergeStateStatus":"CLEAN","mergeable":"MERGEABLE","state":"OPEN"}
=== #321 ===
{"mergeStateStatus":"UNSTABLE","mergeable":"MERGEABLE","pending":["Full test suite (offline, mocked)"],"state":"OPEN"}
[/external_agent_tool_result]
```

#### 162. assistant (2026-07-02T06:11:32.857Z)

#319 is CLEAN and ready. Let me check whether the verification workflow finished (its result is my pre-merge gate):

#### 163. assistant (2026-07-02T06:11:32.857Z)

```
[external_agent_tool_call: Bash]
description: List workflow output dir
command: ls -1 "C:\Users\avidu\claude\projects\C--Users-avidu-Projects-badminton-highlight-
```

```
indexer\73a3938e-1da3-4e28-bfa7-346a15718301\subagents\workflows\wf_753ff171-b1d" 2>&1 | head
-40
[/external_agent_tool_call]
```

#### 164. assistant (2026-07-02T06:11:32.857Z)

```
[external_agent_tool_result]
agent-a687cc70ecfd80619.jsonl
agent-a687cc70ecfd80619.meta.json
agent-a8fc07a323b7c4b46.jsonl
agent-a8fc07a323b7c4b46.meta.json
agent-a9c6b91377cc917c1.jsonl
agent-a9c6b91377cc917c1.meta.json
journal.jsonl
[/external_agent_tool_result]
```

#### 165. assistant (2026-07-02T06:11:32.857Z)

```
[external_agent_tool_call: Bash]
description: Read workflow journal result
command: echo "=== journal tail (final result) ==="; tail -5 "C:\Users\avidu\.claude\projects\C
--Users-avidu-Projects-badminton-highlight-
indexer\73a3938e-1da3-4e28-bfa7-346a15718301\subagents\workflows\wf_753ff171-b1d\journal.jsonl"
2>&1
[/external_agent_tool_call]
```

#### 166. assistant (2026-07-02T06:11:32.857Z)

```
[external_agent_tool_result]
=== journal tail (final result) ===
{"type":"started","key":"v2:6535077d96ce0ec55f80c1bb3601945440739a596e86fbfec8f7bc3a52d5b410","a
gentId":"a9c6b91377cc917c1"}
{"type":"started","key":"v2:b0538960e0467a5b260ceac6f9e4a4ea11f0bfb0d52c03e1339052e0ef24c2a1","a
gentId":"a687cc70ecfd80619"}
{"type":"started","key":"v2:6aad32dd292e3cc206a71641533f73921d9386760370e6d35386361515a85412","a
gentId":"a8fc07a323b7c4b46"}
[/external_agent_tool_result]
```

#### 167. assistant (2026-07-02T06:11:32.857Z)

The verification workflow is still running (3 agents in progress). Merge is gated on it. Meanwhile, the master-breakage investigation is read-only and independent ? let me start understanding what's actually broken on master while the workflow finishes. Let me check master's CI and reproduce the breakage.

#### 168. assistant (2026-07-02T06:11:32.857Z)

```
[external_agent_tool_call: Bash]
description: Check master CI status
command: echo "=== recent CI runs on master ==="; gh run list --branch master --limit 6 --json
conclusion,status,name,headSha,createdAt --jq '.[] | "\(.conclusion //
.status)\t\t\(.name)\t\t\(.headSha[0:8])"' 2>&1 | head -20
echo; echo "=== latest workflow run for master HEAD 48f9d20 ==="; gh run list --branch master
--limit 3 --json databaseId,conclusion,workflowName,headSha 2>&1 | head
[/external_agent_tool_call]
```

#### 169. assistant (2026-07-02T06:11:32.857Z)

```
[external_agent_tool_result]
=== recent CI runs on master ===
success CI 48f9d20c
cancelled CI e221cb92
success CI d301a7fd
success CI fa2342c0
```

```
success CI 7a40434a
success CI 5c2eec81
```

```
=== latest workflow run for master HEAD 48f9d20 ===
[{"conclusion":"success","databaseId":27952730607,"headSha":"48f9d20c29e0a5676fd4c03892e9d7d44ad428b7","workflowName":"CI"}, {"conclusion":"cancelled","databaseId":27952716794,"headSha":"e221cb928ca1595102fbf7ad1bc0e64727bacb35","workflowName":"CI"}, {"conclusion":"success","databaseId":27949441305,"headSha":"d301a7fdb273a6237f39f9cc5e5d436b1c3271cb","workflowName":"CI"}]
[/external_agent_tool_result]
```

#### 170. assistant (2026-07-02T06:11:32.857Z)

Master's CI is actually **green** (commit 48f9d20 CI = success). So "broken" is the **local** full-suite failure I already hit: `test_served_gate_a_regression` (skipped in CI because its corpus manifest is gitignored, but it runs locally). Let me characterize it precisely now ? read the full test and its history ? so I'm ready to fix it properly after the merge.

#### 171. assistant (2026-07-02T06:11:32.857Z)

```
[external_agent_tool_call: Read]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_served_gate_a_regression.py
[/external_agent_tool_call]
```

#### 172. assistant (2026-07-02T06:11:32.857Z)

```
[external_agent_tool_result]
1 """SERVED Gate-A regression litmus ? the *production* FusionSegmenter path on real
footage.
2
3 The flagged gap (#146): the served default flip (`default_segementer=fusion_hybrid` +
4 `min_crossings=2`) is mock-tested only on real video (`test_fusion_hybrid.py`
drives a
5 MagicMock runner). The offline ablation litmus
6 (`test_ablation.test_litmus_reproduces_gate_a`)
7 validates the Gate-A signal ? but on the permissive proposal windowing
8 (`distill_local.PROPOSAL`), NOT the production operating point. This module closes the
gap: it
9 runs the real `FusionSegmenter` decision seams (`enrich_candidate_stats` net-
crossings +
10 `select_for_handoff` Gate A) over windows from the production
11 `trajectory_to_action_windows` config, on the cached golden WASB trajectories, and
asserts
12 the served path's measured behaviour.
13
14 GPU-free / Gemini-free (person cue OFF) ? it stops at the AI-handoff boundary. Skips
cleanly in
15 CI where the golden trajectories are absent; runs on the owner box where they are
present, over
16 whatever golden clips are currently on disk (`golden_specs` requires >= 6).
17
18 CORPUS NOTE ? re-grounded 2026-06-18 (the golden set is mutable and grows over time).
The
19 original 6 single-court golden clips this litmus was first calibrated on (2026-06-14)
have been
20 retired from the box ? their labels are gone ? so the present golden set is the 9
multicourt
21 clips now in `output/` (mahadevpura_1/2, GX0101xx, GX0x0094, Badminton_BXH_2,
Boxhill_Doubles).
22 On that corpus the served path is recall-starved: WASB tracks a single shuttle while
several
23 courts rally at once, so `trajectory_to_action_windows` emits FEWER windows than there
are
24 rallies (served over-seg ratio < 1, i.e. under-segmentation). Because Gate A only ever
DROPS
```

```

24 windows, on this under-segmented corpus it is mildly **F1-negative** (mean served ?F1 ?
-0.034),
25 not the ~neutral -0.008 the original 6 showed. Under the permissive PROPOSAL windowing
the same
26 gate is ~F1-neutral (?F1 ? -0.001) while still cutting over-segmentation (1.16?0.64).
27
28 The canonical ***+0.074 / 6-of-6 / p=0.031** Gate-A win was a property of the
original-6 corpus
29 and is NOT reproduced here (those clips are no longer on the box); it is preserved in
the archive
30 (``docs/archives/quality-iterations/2026-06-first-ab-experiment/``). These tests now pin
the
31 **present-corpus** behaviour so a future windowing/corpus change that silently shifts it
trips the
32 litmus. The pinned magnitudes are a date-stamped snapshot of the present golden set and
will need
33 re-measurement as the corpus grows.
34 ""
35 from __future__ import annotations
36
37 import json
38 import os
39
40 import pytest
41
42 # cv2 is imported transitively by trajectory_hybrid (the served engine). On a box
without it
43 # (CI), skip the whole module rather than erroring at import.
44 pytest.importorskip("cv2")
45
46 from backend.eval import served_gate_a as sga # noqa: E402
47 from backend.pipeline.detectors.tracknet_runner import TrackNetRunner # noqa: E402
48 from backend.pipeline.detectors import rally_gate # noqa: E402
49 from backend.eval.calibrate_wasb import load_golden_gts # noqa: E402
50 from backend.eval.metrics import score # noqa: E402
51 from backend.eval import segmentation_metrics as _segm # noqa: E402
52
53 # The golden manifest lives in the MAIN checkout's output/ (gitignored; absent in a
fresh
54 # worktree / CI). Resolve it relative to this repo root first, with an env override for
an
55 # out-of-tree checkout.
56 _REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
57 MANIFEST = os.environ.get(
58 "SERVED_GATE_A_MANIFEST",
59 os.path.join(_REPO_ROOT, "output", "human_lovo_manifest.json"),
60)
61
62
63 def _golden_specs():
64 """Return the manifest specs whose trajectory + label files are all present, else
[]."""
65 if not os.path.isfile(MANIFEST):
66 ...
[/external_agent_tool_result]

```

### 173. assistant (2026-07-02T06:11:32.857Z)

Now I fully understand the breakage. The test  
(`test\_proposal\_windowing\_gate\_a\_is\_f1\_neutral\_and\_cuts\_over\_seg`) pins **\*\*date-stamped empirical snapshots\*\*** (over-seg `mean\_seg\_off ? 1.16`) measured 2026-06-18 on the local golden corpus. The test's own docstring says these "will need re-measurement as the corpus grows." It got `1.36` because the **\*\*golden-corpus-hygiene work (#316)** changed the local corpus **\*\* ? exactly the kind of shift this litmus is designed to trip on.** CI skips it (manifest gitignored), so master's CI is green; only the local run breaks.



```
(only set/used inside `if args.trajectory_source == 'detector'` at 393/411); eval is a separate
local-FS-only path (blocker #2) not in this worker. (7) History recounting matches
origin/master:DATA_IN_GCS.md:206-213 verbatim in substance (fname.replace,
video_by_stem.get(name) vs bare stems, every clip skipped -> <2 videos abort)."}}
{"type":"result","key":"v2:b0538960e0467a5b260ceac6f9e4a4ea11f0bfb0d52c03e1339052e0ef24c2a1","ag
entId":"a687cc70ecfd80619","result":{"dimension":"Test quality / regression-guarding strength of
PR #319 worker-train tests","ok":true,"blocking_issues":[],"nits":["test_cmd_train_detector_abor
ts_clean_when_fewer_than_two_videos_join PASSES on master too (verified) because master aborts
EVERY clip with rc 2 ? so this test alone does not distinguish fixed-vs-broken. It is not relied
on as the regression guard (the join/skip tests are), and it correctly guards the legitimate
clean-abort path + that subprocess.run is never reached, so this is acceptable. Worth a one-line
comment noting it is not a pre-fix differentiator.", "The join test's rc==0 also depends on the
GPU/harness boundary being faithfully stubbed (real _resolve_train_detector + stub runner write
a real trajectory; only _probe_video_fps_width and subprocess.run are patched). This is the
right seam, but the test would also break if the stub_r...
[/external_agent_tool_result]
```

#### 176. assistant (2026-07-02T06:11:32.857Z)

Now the breakage is fully diagnosed ? and it's revealing. The manifest now resolves \*\*15 clips, not the 9\*\* the test was calibrated on (2026-06-18). The six newly-present clips are exactly the \*\*original-6 single-court clips\*\* (`adarsh\_avi\_singles`, `kushagra\_singles`, `largetest\_doubles`, `gbaaddy`, `testlarge\_short`, `mahadevpura\_singles`) that the docstring said were "retired from the box ? their labels are gone." The golden-corpus-hygiene migration (#316) \*\*restored the original-6 labels+trajectories\*\* from the F: archive. Single-court clips over-segment (~1.68) vs the recall-starved multicourt ones (~0.64), so the blend moved `mean\_seg\_off` 1.16 ? 1.36. \*\*The litmus tripped correctly on a real corpus change ? not a code regression.\*\*

Verification workflow: 2 of 3 agents back, both \*\*ok=true, zero blocking issues\*\* (doc-accuracy confirmed no fabricated claims; test-rigor independently confirmed the tests fail on master pre-fix). Waiting on the 3rd (code-correctness). Let me check its progress and #321 CI.

#### 177. assistant (2026-07-02T06:11:32.857Z)

```
[external_agent_tool_call: Bash]
description: Check 3rd agent and 321 CI
command: echo "=== workflow: 3rd agent result present? ==="; grep -c '"type":"result"'
"C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\73a3938e-1da3-4e28-bfa7-346a15718301\subagents\workflows\wf_753ff171-b1d\journal.jsonl"
2>&1
echo "=== #321 full-suite check ==="; gh pr view 321 --json mergeStateStatus,statusCheckRollup
--jq '{mergeStateStatus, pending: [.statusCheckRollup[] | select((.conclusion //
"PENDING")!="SUCCESS") | .name}}' 2>&1
[/external_agent_tool_call]
```

#### 178. assistant (2026-07-02T06:11:32.857Z)

```
[external_agent_tool_result]
=== workflow: 3rd agent result present? ===
2
=== #321 full-suite check ===
{"mergeStateStatus":"CLEAN","pending":[]}
[/external_agent_tool_result]
```

#### 179. assistant (2026-07-02T06:11:32.857Z)

#321 is now CLEAN too. The 3rd verification agent (code-correctness) is still running. Let me check its progress:

#### 180. assistant (2026-07-02T06:11:32.857Z)

```
[external_agent_tool_call: Bash]
description: Peek at code-correctness agent progress
```

```
command: f="C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\73a3938e-1da3-4e28-bfa7-346a15718301\subagents\workflows\wf_753ff171-b1d\agent-a9c6b91377cc917c1.jsonl"
echo "=== code-correctness agent: last activity ==="; tail -3 "$f" 2>&1 | head -c 1500
[/external_agent_tool_call]
```

#### 181. assistant (2026-07-02T06:11:32.857Z)

```
[external_agent_tool_result]
=== code-correctness agent: last activity ===
{"parentUuid":"154c4a81-9fa1-48af-aba6-24fb13ec533d","isSidechain":true,"agentId":"a9c6b91377cc917c1","message":{"model":"claude-opus-4-8","id":"msg_012g7Gn4ko1Eg5UFeune2fCm","type":"message","role":"assistant","content":[{"type":"tool_use","id":"toolu_01GvkmQxhcjBuWrSkQ8Qy3XA","name":"PowerShell","input":{"command":"git worktree remove C:\\Users\\avidu\\Projects_pr319_review --force 2>&1; git worktree prune; Write-Host \"cleaned\\\", \"description\":\"Remove temporary review worktree\"}, \"caller\":{\"type\":\"direct\"}}], \"stop_reason\":\"tool_use\", \"stop_sequence\":null, \"stop_details\":null, \"usage\":{\"input_tokens\":2, \"cache_creation_input_tokens\":1107, \"cache_read_input_tokens\":48993, \"output_tokens\":236, \"server_tool_use\":{\"web_search_requests\":0, \"web_fetch_requests\":0}, \"service_tier\":\"standard\", \"cache_creation\":{\"ephemeral_1h_input_tokens\":0, \"ephemeral_5m_input_tokens\":1107}, \"inference_geo\":\"not_available\", \"iterations\":[{\"input_tokens\":2, \"output_tokens\":236, \"cache_read_input_tokens\":48993, \"cache_creation_input_tokens\":1107, \"cache_creation\":{\"ephemeral_5m_input_tokens\":1107, \"ephemeral_1h_input_tokens\":0}, \"type\":\"message\"}], \"speed\":\"standard\", \"diagnostics\":null}, \"requestId\":\"req_011CcJJykiMxJX5Yu4gvKZEx\", \"attributionAgent\":\"workflow-subagent\", \"type\":\"assistant\", \"uuid\":\"516ccf38-4590-4dc0-9cd2-9eafdd8078f7\", \"timestamp\":\"2026-06-22T12:37:59.079Z\", \"userType\":\"external\", \"entrypoint\":\"claude-desktop\", \"cwd\":\"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\", \"sessionId\":\"73a3938e-1da3-4e28-bf\"}
[/external_agent_tool_result]
```

#### 182. assistant (2026-07-02T06:11:32.857Z)

The code-correctness agent is finalizing (cleaning up its review worktree). Let me get its verdict:

#### 183. assistant (2026-07-02T06:11:32.858Z)

```
[external_agent_tool_call: Bash]
description: Wait for code-correctness verdict
command: f="C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\73a3938e-1da3-4e28-bfa7-346a15718301\subagents\workflows\wf_753ff171-b1d\journal.jsonl"
python - "$f" <<'PY'
import json, sys, time
f=sys.argv[1]
for _ in range(20):
 res=[json.loads(l) for l in open(f,encoding="utf-8") if '"type":"result"' in l]
 cc=[r for r in res if r.get("agentId")==a9c6b91377cc917c1]
 if cc:
 v=cc[0]["result"]
 print("dimension:", v["dimension"])
 print("ok:", v["ok"])
 print("blocking_issues:", json.dumps(v["blocking_issues"], indent=2))
 print("nits:", json.dumps(v["nits"], indent=2))
 print("evidence:", v["evidence"][:1800])
 break
 time.sleep(3)
else:
 print("still running (no result yet)")
PY
[/external_agent_tool_call]
```

#### 184. assistant (2026-07-02T06:11:32.858Z)

```
[external_agent_tool_result]
dimension: correctness
ok: True
blocking_issues: []
nits: [
 "Helper strips only the LEADING date, so a pathological label like
 `2026-06-22_2026-06-22_foo.rallies.csv` yields `2026-06-22_foo` (inner date kept). This is
 actually correct given the contract (it round-trips to a hypothetical
 `videos/2026-06-22_foo.mp4`), but the implicit assumption that no clip <name> begins `YYYY-MM-
 DD_` is worth a one-line note in the docstring or DATA_IN_GCS naming contract.",
 "Degenerate inputs `2026-06-22_.rallies.csv` and `_.rallies.csv` produce an empty `name`. The
 contract guarantees a non-empty <name>, and an empty stem merely misses the video join and is
 skipped (no crash), so this is theoretical only \u2014 but a defensive guard/skip-with-warning
 on empty `name` would make the helper robust to a malformed bucket object.",
 "A structurally-valid-but-impossible date such as `2026-13-99_` is treated as a date prefix
 (regex is structural, not calendar-validated). Harmless and not producible under the authoring-
 date convention; calendar validation would be overkill here."
]
evidence: backend/worker/___main__.py:23 `import re` present at module level.
backend/worker/___main__.py:268-289 helper: `_LABEL_DATE_PREFIX =
re.compile(r"^\d{4}-\d{2}-\d{2}_")` is ^-anchored; `_label_clip_name` does
`fname.replace('.rallies.csv', '').replace('.csv', '')` then `_LABEL_DATE_PREFIX.sub('', name)`.
cmd_train (lines 407-435): `name = _label_clip_name(fname)` feeds the join
`video_by_stem.get(name)` (411), detector spec `{'name': name,...}` (429), synth traj
`f'{name}_traj.csv` (433) and synth spec (435); `local_label` fetch correctly uses the full
dated `fname` (409) so cross-date same-clip labels don't collide on disk. Adversarial probe (run
against the actual regex): anchored ? `foo_2026-06-22_bar`->unchanged,
`2026-06-22-foo`->unchanged, `20260622_foo`->unchanged; date-stripped correctly ?
`2026-06-22_GX020094`->`GX020094`, `2026-06-21_mahadevpura_1`->`mahadevpura_1`. Legacy
regression proof: for every no-date label (`GX020094_proxy`, `GX010128`, `mahadevpura_1`,
`Boxhill Doubles_proxy`, `mahadevpura_smoke_180s`) new()==old() byte-for-byte ? change is purely
additive. Naming contract LOCKED (docs/DATA_IN_GCS.md:176-179 `labels/<YYYY-MM-
DD>_<name>.rallies.csv`; :141 `videos/<name>.<ext>`), so the date-prefixed-video edge case is
precluded by convention. Verification on a clean detached worktree at origin/fix/worker-train-
dated-label-join: tests/test_worker.py = 29 passed (incl. parametrized helper cases +
dated->bare join regression + orphan-skip + unprobeable/trajless-skip); run_all_tests.py = 1423
passed, 7 skipped; `ruff check backend/worker/___main__.py tests/test_worker.py` = All checks
passed; `mypy backend/worker/___main__.py` = Success, no issues. Branch contains only its own 3
commits vs origin/master (a2c8be0 fix, b3b4952 test, 56c1415 ruff F401 cleanup); no
[/external_agent_tool_result]
```

#### 185. assistant (2026-07-02T06:11:32.858Z)

**\*\*Verification workflow complete: `all\_ok: true`\*\*** ? all 3 dimensions (code-correctness, doc-accuracy, test-rigor) returned ok=true with **\*\*zero blocking issues\*\***. The nits are minor/theoretical (date-prefix edge cases precluded by the locked naming contract; wording). The test-rigor agent independently confirmed via worktrees that the new tests fail on master pre-fix and pass on the fix branch.

Both PRs are CLEAN, the user's conditions are met (coverage 82%?84% worker / 85.59% overall; docs fixed). Merging now ? **\*\*#319 first\*\*** (so the doc's reference resolves to a merged PR), then **\*\*#321\*\***.

#### 186. assistant (2026-07-02T06:11:32.858Z)

```
[external_agent_tool_call: Bash]
description: Merge both PRs (squash)
command: set -e
echo "=== switch off the doc branch to master + update ==="; git checkout master 2>&1 | tail -2;
git pull --ff-only origin master 2>&1 | tail -2
echo; echo "=== merge #319 (code fix) squash + delete branch ==="; gh pr merge 319 --squash
--delete-branch 2>&1 | tail -4
echo; echo "=== merge #321 (doc) squash + delete branch ==="; gh pr merge 321 --squash --delete-
branch 2>&1 | tail -4
[/external_agent_tool_call]
```

#### 187. assistant (2026-07-02T06:11:32.858Z)

```
[external_agent_tool_result]
=== switch off the doc branch to master + update ===
fatal: 'master' is already used by worktree at 'C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/tier1-windowing'
* branch master -> FETCH_HEAD
Already up to date.
```

```
=== merge #319 (code fix) squash + delete branch ===
```

```
=== merge #321 (doc) squash + delete branch ===
failed to run git: fatal: 'master' is already used by worktree at
'C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/tier1-windowing'
[/external_agent_tool_result]
```

#### 188. assistant (2026-07-02T06:11:32.858Z)

The `--delete-branch` cleanup tripped over `master` being checked out in another worktree (`tier1-windowing`). The API merge may still have succeeded. Let me check the actual state of both PRs:

#### 189. assistant (2026-07-02T06:11:32.858Z)

```
[external_agent_tool_call: Bash]
description: Check merge states
command: echo "=== #319 state ==="; gh pr view 319 --json state,mergedAt,mergeCommit --jq
'{state, mergedAt, mergeCommit: .mergeCommit.oid}' 2>&1
echo "=== #321 state ==="; gh pr view 321 --json
state,mergedAt,mergeCommit,mergeable,mergeStateStatus --jq '{state, mergedAt, mergeable,
mergeStateStatus}' 2>&1
echo "=== current branch ==="; git branch --show-current
[/external_agent_tool_call]
```

#### 190. assistant (2026-07-02T06:11:32.858Z)

```
[external_agent_tool_result]
=== #319 state ===
{"mergeCommit":"b72e6de027d6b5f2a35076a4ac259cf7907a62eb","mergedAt":"2026-06-
22T12:39:08Z","state":"MERGED"}
=== #321 state ===
{"mergeStateStatus":"UNKNOWN","mergeable":"UNKNOWN","mergedAt":"2026-06-
22T12:39:12Z","state":"MERGED"}
=== current branch ===
docs/data-gcs-train-blocker-resolved
[/external_agent_tool_result]
```

#### 191. assistant (2026-07-02T06:11:32.858Z)

Both PRs are **MERGED** ? (#319 ? b72e6de at 12:39:08Z; #321 at 12:39:12Z). Only the local `--delete-branch` cleanup failed (master lives in another worktree ? `tier1-windowing`). Let me confirm origin/master has both, prune remote branches, and set up for the master fix.

#### 192. assistant (2026-07-02T06:11:32.858Z)

```
[external_agent_tool_call: Bash]
description: Confirm merges on origin/master, prune
command: git fetch origin --prune 2>&1 | tail -3
echo "=== origin/master top 4 (should include #319 + #321) ==="; git log --oneline -4
origin/master 2>&1
echo "=== remote feature branches still around? ==="; git branch -r 2>&1 | grep -E 'worker-
train-dated|data-gcs-train' || echo "(both remote branches deleted)"
[/external_agent_tool_call]
```

### 193. assistant (2026-07-02T06:11:32.858Z)

```
[external_agent_tool_result]
- [deleted] (none) -> origin/docs/data-gcs-train-blocker-resolved
- [deleted] (none) -> origin/fix/worker-train-dated-label-join
 48f9d20..db74f9b master -> origin/master
=== origin/master top 4 (should include #319 + #321) ===
db74f9b docs(data): mark DATA_IN_GCS $7b blocker #1 resolved by #319 (#321)
b72e6de fix(worker): strip ISO-date prefix when joining labels to videos in train (#319)
48f9d20 feat(nightly): make the Cloud Run scheduler path work E2E + turnkey email/trigger (#320)
e221cb9 docs(data): GCS corpus source-of-truth map + Phase-1 metadata migration (#316)
=== remote feature branches still around? ===
(both remote branches deleted)
[/external_agent_tool_result]
```

### 194. assistant (2026-07-02T06:11:32.858Z)

Both merges are clean on origin/master, remote branches pruned.

Now the master fix. Let me mark the new phase and set up a branch off the freshly-merged master. The fix requires **re-measuring** the eval snapshots on the current 15-clip corpus (this is eval-sensitive owner data, so I measure before changing anything).

### 195. assistant (2026-07-02T06:11:32.858Z)

```
[external_agent_tool_call: mcp_ccd_session__mark_chapter]
input: {"title":"Fix broken master (eval litmus)","summary":"test_served_gate_a_regression
tripped: golden corpus grew 9?15 clips (original-6 restored by #316); re-ground the snapshot"}
[/external_agent_tool_call]
```

### 196. assistant (2026-07-02T06:11:32.858Z)

```
[external_agent_tool_result]
Chapter marked: "Fix broken master (eval litmus)". Continue your current work.
[/external_agent_tool_result]
```

### 197. assistant (2026-07-02T06:11:32.858Z)

```
[external_agent_tool_call: Bash]
description: Create fix branch off updated master
command: echo "=== clean tree? ==="; git status --porcelain | grep -v '^??' || echo "(clean
tracked)"
echo "=== create fix branch off origin/master (has #319+#321) ==="; git checkout -b fix/served-
gate-a-regrounded-15clip origin/master 2>&1 | tail -2
echo "=== HEAD ==="; git log --oneline -1
echo "=== scratch gitignored? ==="; git check-ignore scratch/x 2>&1 && echo "scratch ignored" ||
echo "scratch NOT ignored"
[/external_agent_tool_call]
```

### 198. assistant (2026-07-02T06:11:32.858Z)

```
[external_agent_tool_result]
=== clean tree? ===
(clean tracked)
=== create fix branch off origin/master (has #319+#321) ===
branch 'fix/served-gate-a-regrounded-15clip' set up to track 'origin/master'.
Switched to a new branch 'fix/served-gate-a-regrounded-15clip'
=== HEAD ===
db74f9b docs(data): mark DATA_IN_GCS $7b blocker #1 resolved by #319 (#321)
=== scratch gitignored? ===
scratch/x
scratch ignored
[/external_agent_tool_result]
```

### 199. assistant (2026-07-02T06:11:32.858Z)

Now let me write a measurement script that reproduces every pinned quantity in the litmus on the current 15-clip corpus ? so I re-ground from real measurement, not guesswork:

### 200. assistant (2026-07-02T06:11:32.858Z)

```
[external_agent_tool_call: Write]
file: C:\Users\avidu\Projects\badminton-highlight-
indexer\scratch\measure_served_gate_a_15clip.py
[/external_agent_tool_call]
```

### 201. assistant (2026-07-02T06:11:32.858Z)

```
[external_agent_tool_result]
File created successfully at: C:\Users\avidu\Projects\badminton-highlight-
indexer\scratch\measure_served_gate_a_15clip.py (file state is current in your context ? no need
to Read it back)
[/external_agent_tool_result]
```

### 202. assistant (2026-07-02T06:11:32.858Z)

```
[external_agent_tool_call: Bash]
description: Measure all litmus quantities on 15-clip corpus
command: python scratch/measure_served_gate_a_15clip.py 2>&1 | tail -50
[/external_agent_tool_call]
```

### 203. assistant (2026-07-02T06:11:32.858Z)

```
[external_agent_tool_result]
Traceback (most recent call last):
 File "C:\Users\avidu\Projects\badminton-highlight-
indexer\scratch\measure_served_gate_a_15clip.py", line 12, in <module>
 from backend.eval import served_gate_a as sga
ModuleNotFoundError: No module named 'backend'
[/external_agent_tool_result]
```

### 204. assistant (2026-07-02T06:11:32.859Z)

```
[external_agent_tool_call: Bash]
description: Run measurement with repo root on path
command: PYTHONPATH=. python scratch/measure_served_gate_a_15clip.py 2>&1 | tail -50
[/external_agent_tool_call]
```

### 205. assistant (2026-07-02T06:11:32.859Z)

```
[external_agent_tool_result]
MANIFEST: C:\Users\avidu\Projects\badminton-highlight-indexer\output\human_lovo_manifest.json
resolved specs: 15
```

```
PROPOSAL-WINDOWING per clip (name, n_gt, n_off, n_on, seg_off, seg_on, f1_off, f1_on):
GX020094 gt=40 off=99 on=28 seg_off=2.48 seg_on=0.70 f1_off=0.432 f1_on=0.559
testlarge_short gt=6 off=14 on=10 seg_off=2.33 seg_on=1.67 f1_off=0.300 f1_on=0.375
GX030094 gt=41 off=79 on=42 seg_off=1.93 seg_on=1.02 f1_off=0.350 f1_on=0.361
mahadevpura_singles gt=31 off=57 on=39 seg_off=1.84 seg_on=1.26 f1_off=0.455 f1_on=0.571
adarsh_avi_singles gt=96 off=172 on=84 seg_off=1.79 seg_on=0.88 f1_off=0.597 f1_on=0.722
largetest_doubles gt=49 off=86 on=38 seg_off=1.76 seg_on=0.78 f1_off=0.593 f1_on=0.667
gbaaddy gt=48 off=82 on=47 seg_off=1.71 seg_on=0.98 f1_off=0.231 f1_on=0.274
GX010137 gt=34 off=56 on=25 seg_off=1.65 seg_on=0.74 f1_off=0.689 f1_on=0.746
Boxhill_Doubles gt=43 off=59 on=50 seg_off=1.37 seg_on=1.16 f1_off=0.235 f1_on=0.258
GX010128 gt=52 off=51 on=43 seg_off=0.98 seg_on=0.83 f1_off=0.291 f1_on=0.316
GX010141 gt=29 off=26 on=13 seg_off=0.90 seg_on=0.45 f1_off=0.400 f1_on=0.143
Badminton_BXH_2 gt=44 off=29 on=24 seg_off=0.66 seg_on=0.55 f1_off=0.027 f1_on=0.029
kushagra_singles gt=32 off=20 on=17 seg_off=0.62 seg_on=0.53 f1_off=0.154 f1_on=0.163
```

```
mahadevpura_2 gt=40 off=14 on=14 seg_off=0.35 seg_on=0.35 f1_off=0.074 f1_on=0.074
mahadevpura_1 gt=10 off=1 on=0 seg_off=0.10 seg_on=0.00 f1_off=0.000 f1_on=0.000
```

```
[proposal] n=15
[proposal] mean_delta (f1_on - f1_off) = +0.0287 (was pinned ~0.0 ±0.03)
[proposal] mean_seg_off = 1.3640 (was pinned 1.16 ±0.10)
[proposal] mean_seg_on = 0.7919 (was pinned 0.64 ±0.10)
[proposal] DURABLE invariant mean_seg_on < mean_seg_off : True (0.792 < 1.364)
```

SERVED-PATH aggregate:

```
[served] mean_delta_f1 = -0.0233 (band pinned [-0.06, 0.03])
[served] mean_f1_on = 0.2134
[served] mean_f1_off = 0.2366
[served] f1_on/f1_off = 0.902 (must be >= 0.5)
[served] mean_seg_on = 0.4696
[served] mean_seg_off = 0.5825
[served] DURABLE seg_on <= seg_off : True
[served] per-video seg monotone (on<=off): True
[served] per-video Gate A only drops (off>=on handoffs): True
[served] num videos: 15 (all n_gt>0: True)
[/external_agent_tool_result]
```

## 206. assistant (2026-07-02T06:11:32.859Z)

Now I have the complete re-measured picture. Critically:

- **All durable invariants hold** (`mean_seg_on < mean_seg_off`, Gate A only drops windows, per-video monotone) ? this is a **corpus change**, not a code regression.
- Only the **proposal-windowing test's absolute snapshots** broke: `mean_seg_off` 1.16?`1.36`, `mean_seg_on` 0.64?`0.79`. The served-path tests still pass (they use ranges).
- A real finding: with the original-6 single-court clips back, the proposal-windowing gate is now mildly **F1-positive** (?F1 +0.029, was ?-0.001 on 9 clips) ? the archived +0.074 story partially re-entering the blend.

The fix = honest re-grounding to the 15-clip corpus (the test's docstring explicitly says snapshots "will need re-measurement as the corpus grows"). Let me update the snapshots **and** the now-stale narrative/comments (honesty matters more than minimalism ? a passing-but-lying test is worse).

## 207. assistant (2026-07-02T06:11:32.859Z)

```
[external_agent_tool_call: Edit]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_served_gate_a_regression.py
[/external_agent_tool_call]
```

## 208. assistant (2026-07-02T06:11:32.859Z)

```
[external_agent_tool_result]
The file C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_served_gate_a_regression.py has been updated successfully. (file state is current in your context ? no need to Read it back)
[/external_agent_tool_result]
```

## 209. assistant (2026-07-02T06:11:32.859Z)

```
[external_agent_tool_call: Edit]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_served_gate_a_regression.py
[/external_agent_tool_call]
```

## 210. assistant (2026-07-02T06:11:32.859Z)

```
[external_agent_tool_result]
The file C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_served_gate_a_regression.py has been updated successfully. (file state is
```

current in your context ? no need to Read it back)  
[/external\_agent\_tool\_result]

**211. assistant (2026-07-02T06:11:32.859Z)**

Now the proposal-windowing test (docstring + the failing snapshot assertions):

**212. assistant (2026-07-02T06:11:32.859Z)**

[external\_agent\_tool\_call: Edit]  
file: C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test\_served\_gate\_a\_regression.py  
[/external\_agent\_tool\_call]

**213. assistant (2026-07-02T06:11:32.859Z)**

[external\_agent\_tool\_result]  
The file C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test\_served\_gate\_a\_regression.py has been updated successfully. (file state is current in your context ? no need to Read it back)  
[/external\_agent\_tool\_result]

**214. assistant (2026-07-02T06:11:32.859Z)**

[external\_agent\_tool\_call: Edit]  
file: C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test\_served\_gate\_a\_regression.py  
[/external\_agent\_tool\_call]

**215. assistant (2026-07-02T06:11:32.859Z)**

[external\_agent\_tool\_result]  
The file C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test\_served\_gate\_a\_regression.py has been updated successfully. (file state is current in your context ? no need to Read it back)  
[/external\_agent\_tool\_result]

**216. assistant (2026-07-02T06:11:32.859Z)**

Now run the full litmus module to confirm all 5 tests pass on the 15-clip corpus:

**217. assistant (2026-07-02T06:11:32.859Z)**

[external\_agent\_tool\_call: Bash]  
description: Run the re-grounded litmus module  
command: python -m pytest tests/test\_served\_gate\_a\_regression.py -v 2>&1 | tail -20  
[/external\_agent\_tool\_call]

**218. assistant (2026-07-02T06:11:32.859Z)**

[external\_agent\_tool\_result]  
===== test session starts =====  
platform win32 -- Python 3.13.13, pytest-9.0.3, pluggy-1.6.0 --  
C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe  
cachedir: .pytest\_cache  
rootdir: C:\Users\avidu\Projects\badminton-highlight-indexer  
configfile: pytest.ini  
plugins: anyio-4.13.0, cov-7.1.0  
collecting ... collected 5 items  
  
tests/test\_served\_gate\_a\_regression.py::test\_served\_path\_runs\_on\_all\_golden\_videos PASSED [ 20%]  
tests/test\_served\_gate\_a\_regression.py::test\_served\_gate\_a\_does\_not\_materially\_regress\_f1 PASSED [ 40%]  
tests/test\_served\_gate\_a\_regression.py::test\_served\_gate\_a\_does\_not\_increase\_over\_segmentation PASSED [ 60%]

```
tests/test_served_gate_a_regression.py::test_served_uses_production_windowing_operating_point
PASSED [80%]
tests/test_served_gate_a_regression.py::test_proposal_windowing_gate_a_is_f1_neutral_and_cuts_ov
er_seg PASSED [100%]
```

```
===== 5 passed in 20.02s =====
[/external_agent_tool_result]
```

#### 219. assistant (2026-07-02T06:11:32.859Z)

All 5 litmus tests pass. Let me run the full suite to confirm master is unbroken locally, and check for any stray references to the old 9-clip narrative elsewhere:

#### 220. assistant (2026-07-02T06:11:32.859Z)

```
[external_agent_tool_call: Grep]
input: {"pattern":"9 multicourt|9-clip|present 9|retired original-6|those clips are no
longer","output_mode":"files_with_matches"}
[/external_agent_tool_call]
```

#### 221. assistant (2026-07-02T06:11:32.859Z)

```
[external_agent_tool_result]
Found 2 files
tests\test_served_gate_a_regression.py
docs\QUALITY_ITERATIONS.md
[/external_agent_tool_result]
```

#### 222. assistant (2026-07-02T06:11:32.859Z)

Let me run the full suite (confirming master is unbroken) in the background while I check what `QUALITY\_ITERATIONS.md` says about the 9-clip set:

#### 223. assistant (2026-07-02T06:11:32.859Z)

```
[external_agent_tool_call: Bash]
description: Full suite after the fix
command: python run_all_tests.py --cov=backend --cov=training --cov-fail-under=70 2>&1 | tail -6
[/external_agent_tool_call]
```

#### 224. assistant (2026-07-02T06:11:32.859Z)

```
[external_agent_tool_result]
Command running in background with ID: bqku4e6i9. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\73a3938e-1da3-4e28-bfa7-346a15718301\tasks\bqku4e6i9.output. You will be notified when
it completes. To check interim output, use Read on that file path.
[/external_agent_tool_result]
```

#### 225. assistant (2026-07-02T06:11:32.859Z)

```
[external_agent_tool_call: Grep]
input: {"pattern":"9 multicourt|9-clip|present 9|retired original-6|those clips are no longer|1\
\\.16|0\\.64","path":"docs/QUALITY_ITERATIONS.md","output_mode":"content","-n":true,"-C":2}
[/external_agent_tool_call]
```

#### 226. assistant (2026-07-02T06:11:32.860Z)

```
[external_agent_tool_result]
588-
589-> **Re-grounded 2026-06-18.** The original-6 single-court golden clips this litmus was
calibrated on have been
590:> retired from the box (their labels are gone); the present golden set is **9 multicourt
```

clips\*\*. On that  
591-> recall-starved corpus (WASB tracks one shuttle while several courts rally ? served over-seg ratio < 1) the  
592-> served Gate-A ?F1 is now **\*\*?0.034\*\*** (was ?0.008), and under the permissive proposal windowing the gate is  
593:> **\*\*~F1-neutral\*\*** (?F1 ? ?0.001) while still cutting over-seg (1.16?0.64) ? it no longer reproduces the +0.074  
594-> live. The canonical **\*\*+0.074 / 6-of-6 / p=0.031\*\*** win remains a real result of the original-6 corpus,  
595-> preserved in the [first-A/B archive](archives/quality-iterations/2026-06-first-ab-experiment/README.md).  
[/external\_agent\_tool\_result]

## 227. assistant (2026-07-02T06:11:32.860Z)

`docs/QUALITY\_ITERATIONS.md` carries the **\*\*same now-stale 9-clip narrative\*\*** (the ablation backbone doc, which the owner keeps as "honest measured results only"). Leaving it stale would contradict the re-grounded test and be dishonest. Let me read its context and update it to match the 15-clip reality:

## 228. assistant (2026-07-02T06:11:32.860Z)

[external\_agent\_tool\_call: Read]  
file: C:\Users\avidu\Projects\badminton-highlight-indexer\docs\QUALITY\_ITERATIONS.md  
[/external\_agent\_tool\_call]

## 229. assistant (2026-07-02T06:11:32.860Z)

[external\_agent\_tool\_result]  
576 - `backend/config/models.py` ? `FusionConfig.min\_crossings` default **\*\*2 ? 0\*\*** (Gate A **\*\*OFF\*\*** by default).  
577 - Tests (`tests/test\_config.py`, `tests/test\_fusion\_hybrid.py`) restored to the Gate-A-OFF defaults; the  
578 **\*\*persisted-superset invariant is preserved\*\*** (gating only ever filtered the AI handoff, never persistence).  
579 - `eval\_baselines/experiment\_leaderboard.json` ? the #146 `promotion` record carries a `reverted (eval?serve)`  
580 note.  
581  
582 **\*\*Gate A is NOT deleted ? it remains opt-in\*\*** (`indexing.fusion.min\_crossings: 2`) for footage with more  
583 held-shuttle/service-feed false positives than the golden 6, and the offline ablation evidence below stands  
584 (it is a real proposal-windowing win). The path back to serving it is to **\*\*align the served windowing toward**  
585 **the proposal operating point\*\*** and re-run the #148 litmus to confirm the lift transfers. The #148 litmus is  
586 now the standing guard: it pins the honest measured behaviour on the **\*\*present golden corpus\*\*** and trips if a  
587 future windowing/corpus change silently shifts it.  
588  
589 > **\*\*Re-grounded 2026-06-18.\*\*** The original-6 single-court golden clips this litmus was calibrated on have been  
590 > retired from the box (their labels are gone); the present golden set is **\*\*9 multicourt clips\*\***. On that  
591 > recall-starved corpus (WASB tracks one shuttle while several courts rally ? served over-seg ratio < 1) the  
592 > served Gate-A ?F1 is now **\*\*?0.034\*\*** (was ?0.008), and under the permissive proposal windowing the gate is  
593 > **\*\*~F1-neutral\*\*** (?F1 ? ?0.001) while still cutting over-seg (1.16?0.64) ? it no longer reproduces the +0.074  
594 > live. The canonical **\*\*+0.074 / 6-of-6 / p=0.031\*\*** win remains a real result of the original-6 corpus,  
595 > preserved in the [first-A/B archive](archives/quality-iterations/2026-06-first-ab-

```
experiment/README.md).
596
597 ### Acting on the first A/B: Gate-A PROMOTED + learned variant FIXED + GX010125
corroboration *(2026-06-14)*
598 **Gate-A (net-crossings ? 2) ? PROMOTED (first cue promoted via the harness).** The
measured win
599 (?F1 +0.074, **6/6 golden matches, sign-test p=0.031**, CI [+0.042,+0.104]; over-seg
1.68?1.01 ? the
600 [first-A/B archive](archives/quality-iterations/2026-06-first-ab-experiment/README.md))
clears the honest
601 bar. **Recommended config:** `indexing.fusion.min_crossings = 2` on the fusion path (the
eval-only Gate A
[/external_agent_tool_result]
```

### 230. assistant (2026-07-02T06:11:32.860Z)

```
[external_agent_tool_call: Edit]
file: C:\Users\avidu\Projects\badminton-highlight-indexer\docs\QUALITY_ITERATIONS.md
[/external_agent_tool_call]
```

### 231. assistant (2026-07-02T06:11:32.860Z)

```
[external_agent_tool_result]
The file C:\Users\avidu\Projects\badminton-highlight-indexer\docs\QUALITY_ITERATIONS.md has been
updated successfully. (file state is current in your context ? no need to Read it back)
[/external_agent_tool_result]
```

### 232. assistant (2026-07-02T06:11:32.860Z)

Now let me lint the changed test file while the full suite finishes in the background:

### 233. assistant (2026-07-02T06:11:32.860Z)

```
[external_agent_tool_call: Bash]
description: Ruff on test file; check suite progress
command: echo "=== ruff on changed test file ==="; ruff check
tests/test_served_gate_a_regression.py 2>&1 | tail -5
echo "=== full suite background status ==="; tail -4 "C:\Users\avidu\AppData\Local\Temp\claude\C
--Users-avidu-Projects-badminton-highlight-
indexer\73a3938e-1da3-4e28-bfa7-346a15718301\tasks\bqku4e6i9.output" 2>&1
[/external_agent_tool_call]
```

### 234. assistant (2026-07-02T06:11:32.860Z)

```
[external_agent_tool_result]
=== ruff on changed test file ===
All checks passed!
=== full suite background status ===
[/external_agent_tool_result]
```

### 235. assistant (2026-07-02T06:11:32.860Z)

Ruff clean. The full suite is still running in the background ? let me wait for it to confirm the previously-failing test is now green and nothing else regressed before I commit. Let me check its progress:

### 236. assistant (2026-07-02T06:11:32.860Z)

```
[external_agent_tool_call: Bash]
description: Wait for full suite to finish
command: sleep_done=0; for i in $(seq 1 40); do if grep -q "RESULT:"
"C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\73a3938e-1da3-4e28-bfa7-346a15718301\tasks\bqku4e6i9.output" 2>/dev/null; then
sleep_done=1; break; fi; sleep 5; done; tail -8 "C:\Users\avidu\AppData\Local\Temp\claude\C--
```

```
Users-avidu-Projects-badminton-highlight-
indexer\73a3938e-1da3-4e28-bfa7-346a15718301\tasks\bqku4e6i9.output" 2>&1
[/external_agent_tool_call]
```

### 237. assistant (2026-07-02T06:11:32.860Z)

```
[external_agent_tool_result]
TOTAL 10001 1441 86%
Required test coverage of 70% reached. Total coverage: 85.59%
1430 passed, 7 warnings in 148.51s (0:02:28)
=====
RESULT: PASS - all tests green
=====
[/external_agent_tool_result]
```

### 238. assistant (2026-07-02T06:11:32.860Z)

**\*\*RESULT: PASS ? all tests green.\*\*** 1430 passed (previously 1429 passed + 1 failed), coverage 85.59%. The litmus is re-grounded and master is unbroken locally.

Let me commit the fix (test + doc), verifying branch hygiene:

### 239. assistant (2026-07-02T06:11:32.860Z)

```
[external_agent_tool_call: Bash]
description: Commit the litmus re-grounding
command: git add tests/test_served_gate_a_regression.py docs/QUALITY_ITERATIONS.md
echo "=== staged (only these 2) ==="; git diff --cached --name-only
echo "=== branch + commits ahead of origin/master ==="; git branch --show-current; git log
--oneline origin/master..HEAD | head
git commit -F - <<'EOF'
test(eval): re-ground served Gate-A litmus to the 15-clip golden corpus
```

The served Gate-A litmus (tests/test\_served\_gate\_a\_regression.py) pinned date-stamped over-segmentation snapshots measured 2026-06-18 on a 9-clip multicourt golden set. The GCS corpus migration (#316) RESTORED the original-6 single-court labels + trajectories to the box, so the local golden set is now 15 clips (9 multicourt + 6 single-court). The litmus correctly TRIPPED on this corpus change (its docstring says the snapshots "will need re-measurement as the corpus grows"): mean\_seg\_off 1.16 ? 1.36, mean\_seg\_on 0.64 ? 0.79.

Verified this is a CORPUS change, not a code regression: all durable invariants still hold (gate only ever drops windows; mean\_seg\_on < mean\_seg\_off; per-video monotone; served seg\_on <= seg\_off). Re-measured every pinned quantity on the 15 clips and updated the snapshots + the now-stale 9-clip narrative in the test docstring and docs/QUALITY\_ITERATIONS.md.

Notable measured finding: with the original-6 (which carried the archived +0.074 Gate-A win) back in the blend, the proposal-windowing gate is now mildly F1-POSITIVE (?F1 ? +0.029, was ? -0.001 on the 9-clip set); served path stays mildly negative (?F1 ? -0.023). Bands re-centered on the measured values.

This test SKIPS in CI (golden manifest is gitignored/absent); it runs on the owner box. Re-measurement done locally; the eval narrative re-grounding merits an owner eyeball.

```
Co-Authored-By: Claude Opus 4.8 <noreply@anthropic.com>
EOF
echo "=== committed ==="; git log --oneline -1
[/external_agent_tool_call]
```

### 240. assistant (2026-07-02T06:11:32.860Z)

```
[external_agent_tool_result]
```

```
warning: in the working copy of 'docs/QUALITY_ITERATIONS.md', LF will be replaced by CRLF the
next time Git touches it
=== staged (only these 2) ===
docs/QUALITY_ITERATIONS.md
tests/test_served_gate_a_regression.py
=== branch + commits ahead of origin/master ===
fix/served-gate-a-regrounded-15clip
[fix/served-gate-a-regrounded-15clip 5edd9f8] test(eval): re-ground served Gate-A litmus to the
15-clip golden corpus
 2 files changed, 50 insertions(+), 40 deletions(-)
=== committed ===
5edd9f8 test(eval): re-ground served Gate-A litmus to the 15-clip golden corpus
[/external_agent_tool_result]
```

#### 241. assistant (2026-07-02T06:11:32.860Z)

```
[external_agent_tool_call: Bash]
description: Push the fix branch
command: git push -u origin fix/served-gate-a-regrounded-15clip 2>&1 | tail -4
[/external_agent_tool_call]
```

#### 242. assistant (2026-07-02T06:11:32.860Z)

```
[external_agent_tool_result]
remote:
branch 'fix/served-gate-a-regrounded-15clip' set up to track 'origin/fix/served-gate-a-
regrounded-15clip'.
To https://github.com/Khelsutra/badminton-highlight-indexer.git
 * [new branch] fix/served-gate-a-regrounded-15clip -> fix/served-gate-a-regrounded-15clip
[/external_agent_tool_result]
```

#### 243. assistant (2026-07-02T06:11:32.860Z)

```
[external_agent_tool_call: Bash]
description: Open the litmus re-grounding PR
command: gh pr create --base master --head fix/served-gate-a-regrounded-15clip \
 --title "test(eval): re-ground served Gate-A litmus to the 15-clip golden corpus" \
 --body "$(cat <<'EOF'
```

### What broke

Running the full local suite on `master` fails one test:

```
...
tests/test_served_gate_a_regression.py::test_proposal_windowing_gate_a_is_f1_neutral_and_cuts_ov
er_seg
 assert mean_seg_off == 1.16 ± 0.10 -> got 1.3640
...
```

CI is unaffected ? this litmus **skips** in CI (its golden manifest `output/human\_lovo\_manifest.json` is gitignored/absent); it only runs on the owner box, so `master`'s CI stayed green while the local run broke.

### Why (diagnosed, not guessed)

The litmus pinned **date-stamped over-seg snapshots** measured 2026-06-18 on a **9-clip multicourt** golden set. The **GCS corpus migration (#316)** RESTORED the **original-6 single-court** labels + trajectories to the box (`adarsh\_avi\_singles`, `kushagra\_singles`, `largetest\_doubles`, `gbaaddy`, `testlarge\_short`, `mahadevpura\_singles`), so the local golden set is now **15 clips** (9 multicourt + 6 single-court). Single-court clips over-segment (~1.7) vs the recall-starved multicourt ones (~0.5), so the blend moved `mean\_seg\_off` 1.16 ? 1.36. **The litmus tripped exactly as designed** ? its docstring says the snapshots "will need re-measurement as the corpus grows."

**\*\*Confirmed it's a corpus change, not a code regression:\*\*** every *\*durable\** invariant still holds ? Gate A only ever drops windows, ``mean_seg_on < mean_seg_off``, per-video seg monotone, served ``seg_on <= seg_off``.

## The fix

Re-measured **every** pinned quantity on the current 15 clips and re-grounded the snapshots + the now-stale 9-clip narrative (in the test docstring and ``docs/QUALITY_ITERATIONS.md``):

```
| quantity | old (9-clip, 2026-06-18) | new (15-clip, 2026-06-22) |
|---|---|---|
| proposal `mean_seg_off` | 1.16 | **1.36** |
| proposal `mean_seg_on` | 0.64 | **0.79** |
| proposal `mean_delta` (?F1) | ? ?0.001 (neutral) | **+0.029 (mildly positive)** |
| served `mean_delta_f1` | ? ?0.034 | **?0.023** (band `[-0.06, 0.03]` unchanged, still holds) |
```

**\*\*Notable finding:\*\*** with the original-6 (which carried the archived **\*\*+0.074 / 6-of-6 / p=0.031\*\*** Gate-A win) back in the blend, the proposal-windowing gate is now mildly **\*\*F1-positive\*\*** again, while the served operating point stays mildly negative ? reinforcing the litmus's thesis that the served behaviour is a property of the operating point + corpus, not a broken gate.

## Verification

- ``tests/test_served_gate_a_regression.py`` ? **\*\*5 passed\*\*** on the 15-clip corpus.
- ``python run_all_tests.py --cov=backend --cov=training --cov-fail-under=70`` ? **\*\*1430 passed\*\***, coverage **\*\*85.59%\*\***.
- ``ruff check tests/test_served_gate_a_regression.py`` ? clean.

## ?? For owner review (not auto-merged)

This re-grounds **eval-quality snapshots** on your golden corpus, and **CI cannot independently verify it** (the test skips in CI). The measured numbers above were taken on the box's current 15-clip manifest. Please eyeball the re-grounded narrative/magnitudes before merge ? flagging per the launch-report / honest-measured-results conventions. The corpus growing 9 ? 15 (original-6 restored) also affects other eval narratives, which may be worth a wider sweep.

? Generated with [Claude Code](https://claude.com/claude-code)  
EOF  
)" 2>&1 | tail -3  
[/external\_agent\_tool\_call]

### 244. assistant (2026-07-02T06:11:32.860Z)

```
[external_agent_tool_result]
https://github.com/Khelsutra/badminton-highlight-indexer/pull/324
[/external_agent_tool_result]
```

### 245. assistant (2026-07-02T06:11:32.860Z)

PR **\*\*[#324](https://github.com/Khelsutra/badminton-highlight-indexer/pull/324)\*\*** opened ? deliberately **\*\*not auto-merged\*\*** (it re-grounds eval-quality snapshots and CI can't verify it; left for owner review per the launch-report/honest-results conventions).

Let me record this corpus-state change as a memory (it affects eval beyond this one test) and update the live current-state checkpoint.

### 246. assistant (2026-07-02T06:11:32.860Z)

```
[external_agent_tool_call: Write]
file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\golden-corpus-grew-9-to-15.md
[/external_agent_tool_call]
```

#### 247. assistant (2026-07-02T06:11:32.860Z)

```
[external_agent_tool_result]
File created successfully at: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\golden-corpus-grew-9-to-15.md (file state is current in your context ? no need to Read it back)
[/external_agent_tool_result]
```

#### 248. assistant (2026-07-02T06:11:32.861Z)

```
[external_agent_tool_call: Read]
file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\current-state.md
[/external_agent_tool_call]
```

#### 249. assistant (2026-07-02T06:11:32.861Z)

```
[external_agent_tool_result]
1 ---
2 name: current-state
3 description: "Live micro-checkpoint state for Claude ? updated every checkpoint (not only on request). What just landed, what's next, open decisions."
4 metadata:
5 node_type: memory
6 type: project
7 originSessionId: b13673cc-f087-43b1-9527-35dc0b67b301
8 ---
9
10 # Current state ? updated continuously (see [[working-agreement-checkpoints]])
11
12 > **Scope of this file = LIVE only:** the current handoff + *today's* checkpoints + open PRs/decisions. Keep it read-first-short (?1 page). Older checkpoints are archived in [[current-state-archive]] ? when today's checkpoints age out (next clean-slate day), move them there, don't let them pile up here.
13
14 **Checkpoint:** 2026-06-22 (**NIGHTLY E2E ? Cloud Run scheduler path FIXED + cloud-validated, [#320] MERGED**). master `48f9d20`. Owner asked Option A (make `register_scheduler.sh` actually work) + a trigger command + email setup. **Two bugs caught by an in-cloud dry-run validation** (build the launcher image ? temp dry-run Cloud Run job ? exec): (1) the launcher container is `COPY . /app` but Cloud Build excludes `.git` ? `launch.sh`'s `git archive` failed ? **fixed**: `launch.sh` now tars the baked tree when no `.git` + takes the SHA from `RALLY_GIT_SHA` (+ a `RALLY_DRY_RUN` smoke mode); (2) `gcloud builds submit --tag` has **no `-f` flag** ? added `cloudbuild.launcher.yaml` + `--config`. Also: `register_scheduler.sh` auto-grants the SA IAM (compute.instanceAdmin.v1+iam.serviceAccountUser+run.invoker) + bakes the SHA + `--run-now`; new **`setup_email.sh`** (Gmail app-pw via STDIN ? Secret Manager + SA secretAccessor). **VALIDATED in-cloud**: image built, the container ran launch.sh (no .git ? tar, SHA=4051ed5), staged the tarball, hit "Skipping VM create" (dry-run) ? with the earlier real-L4 run, the FULL Cloud Scheduler?Cloud Run job?launch.sh?ephemeral-GPU-VM chain is proven. Temp job+image deleted; nightly bucket kept; GCP $0. *** Owner go-live (all from a LOCAL master checkout, NOT the droplet): `create_nightly_bucket.sh` (done) ? upload clips/labels to `gs://??/nightly-inputs/` ? `setup_email.sh` ? `register_scheduler.sh --run-now`. Start on demand: `gcloud run jobs execute nightly-regression-launcher --region asia-south1`. ** [[gcp-vm-always-delete]] [[working-agreement-merging]]
15
16 **Checkpoint:** 2026-06-22 (**NIGHTLY E2E ? FAITHFUL WASB run VALIDATED on a real GCP L4 GPU; follow-up [#318]**). Owner: "use GCP GPU VMs (auth/tooling available)." Spun up an **L4** (`rally-nightly-val`, asia-south1-b) via `create_gpu_vm.sh`, set up the real WASB-native conda env + weights (`gs://khelsutra-rally-corpus/models/wasb_badminton_best.pth.tar`), ran `nightly_reelcount.py` with `wasb_hybrid`/`detector_impl=native`/`skip_ai_handoff=true` on `gs://??/videos/mahadevpura_smoke_180s.mp4`: **22 reels, DETERMINISTIC across 3 runs, clean PASS (exit 0), faked regression caught (exit 1, `reel_count 25?22`), cost from real uptime ($0.069/?5.77, 352s @ $0.71/hr)**. **VM DELETED** ($0 idle, no orphan disks ? note `teardown.sh` defaults name `rally-gpu`; set `RALLY_VM_NAME` or `gcloud delete` directly). The real gs://+GPU+WASB run caught **4 deploy bugs the droplet/mocks couldn't**, all fixed in **[#318]** (CI green, merging): (1) `gs://` fetch needs a `dst` (GCS no-passthrough) ? new `_fetch_to`
```

downloads remote into the workdir (+test, also `\_load\_gts`); (2) `run\_on\_vm.sh` must export `WASB\_REPO\_DIR`+`WASB\_PYTHON` (not just `WASB\_WEIGHTS`); (3) weights default `weights/`?`models/`; (4) manifest `skip\_ai\_handoff=true` (pure-engine, no paid Gemini ? deterministic+free). GCP auth = `gcloud` as `avi.dullu@gmail.com`/`gen-lang-client-0306026826`, GPU quota=1, weights in `models/`, clips+labels+cached-trajectories in the bucket. Suite 1413?.  
\*\*? Runner+orchestration now GPU-proven; only the OWNER deploy steps (GCS upload + SMTP secret + `register\_scheduler.sh`) remain to go live.\*\* [[gcp-vm-always-delete]] [[compute-stack-gcp-only]] Owner: "use GCP GPU VM...  
[/external\_agent\_tool\_result]

**250. assistant (2026-07-02T06:11:32.861Z)**

Let me prepend a concise checkpoint to current-state and add the memory index pointer:

**251. assistant (2026-07-02T06:11:32.861Z)**

[external\_agent\_tool\_call: Edit]  
file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\current-state.md  
[/external\_agent\_tool\_call]

**252. assistant (2026-07-02T06:11:32.861Z)**

[external\_agent\_tool\_result]  
The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\current-state.md has been updated successfully. (file state is current in your context ? no need to Read it back)  
[/external\_agent\_tool\_result]

**253. assistant (2026-07-02T06:11:32.861Z)**

Now add the index pointer in MEMORY.md:

**254. assistant (2026-07-02T06:11:32.861Z)**

[external\_agent\_tool\_call: Edit]  
file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\MEMORY.md  
[/external\_agent\_tool\_call]

**255. assistant (2026-07-02T06:11:32.861Z)**

[external\_agent\_tool\_result: error]  
<tool\_use\_error>File has not been read yet. Read it first before writing to it.</tool\_use\_error>  
[/external\_agent\_tool\_result]

**256. assistant (2026-07-02T06:11:32.861Z)**

[external\_agent\_tool\_call: Read]  
file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\MEMORY.md  
[/external\_agent\_tool\_call]

**257. assistant (2026-07-02T06:11:32.861Z)**

[external\_agent\_tool\_result]  
55 - [Sibling internship repo](sibling-internship-repo.md) ? khelacharya shot-intelligence layer builds ON TOP of our rally segmentation; kept off master at branch level (NOT a separate GitHub repo ? corrected 2026-06-06)  
56 - [Paper co-author aim](paper-coauthor-aim.md) ? \*\*standing side-aim\*\*: co-author a paper on "local, cheap & efficient rally detection via multiple clever ideas". Keep `docs/QUALITY\_ITERATIONS.md` as the ablation-ready empirical backbone; honest measured results ONLY.

57

[/external\_agent\_tool\_result]

**258. assistant (2026-07-02T06:11:32.861Z)**

[external\_agent\_tool\_call: Grep]

input: {"pattern":"Eval dual-set regression","path":"C:\\Users\\avidu\\.claude\\projects\\C--Users-avidu-Projects-badminton-highlight-indexer\\memory\\MEMORY.md","output\_mode":"content","-n":true}

[/external\_agent\_tool\_call]

**259. assistant (2026-07-02T06:11:32.861Z)**

[external\_agent\_tool\_result]

42:- [Eval dual-set regression](eval-dual-set-regression.md) ? owner directive (2026-06-16, proven practice): keep a **RAW eval set** (currently-bad videos, random sample) + a **GROWING GOLDEN-finalized eval set** (promote raw?golden when at-par) + a separate **golden TRAINING set**; every quality launch must NOT regress on EITHER eval set (don't lose a graduated win). The **ablation layer A/B**'s these. Checkpointed video folder; wire dual-set aggregate into `run\_regression.py` when R2 lands.

[/external\_agent\_tool\_result]

**260. assistant (2026-07-02T06:11:32.861Z)**

[external\_agent\_tool\_call: Read]

file: C:\\Users\\avidu\\.claude\\projects\\C--Users-avidu-Projects-badminton-highlight-indexer\\memory\\MEMORY.md

[/external\_agent\_tool\_call]

**261. assistant (2026-07-02T06:11:32.861Z)**

[external\_agent\_tool\_result]

42 - [Eval dual-set regression](eval-dual-set-regression.md) ? owner directive (2026-06-16, proven practice): keep a **RAW eval set** (currently-bad videos, random sample) + a **GROWING GOLDEN-finalized eval set** (promote raw?golden when at-par) + a separate **golden TRAINING set**; every quality launch must NOT regress on EITHER eval set (don't lose a graduated win). The **ablation layer A/B**'s these. Checkpointed video folder; wire dual-set aggregate into `run\_regression.py` when R2 lands.

[/external\_agent\_tool\_result]

**262. assistant (2026-07-02T06:11:32.861Z)**

[external\_agent\_tool\_call: Edit]

file: C:\\Users\\avidu\\.claude\\projects\\C--Users-avidu-Projects-badminton-highlight-indexer\\memory\\MEMORY.md

[/external\_agent\_tool\_call]

**263. assistant (2026-07-02T06:11:32.861Z)**

[external\_agent\_tool\_result]

The file C:\\Users\\avidu\\.claude\\projects\\C--Users-avidu-Projects-badminton-highlight-indexer\\memory\\MEMORY.md has been updated successfully. (file state is current in your context ? no need to Read it back)

[/external\_agent\_tool\_result]

**264. assistant (2026-07-02T06:11:32.861Z)**

Let me confirm PR #324's CI (the litmus skips in CI, so it should pass cleanly):

**265. assistant (2026-07-02T06:11:32.861Z)**

[external\_agent\_tool\_call: Bash]

description: Check #324 CI

```

command: sleep_n=0; for i in $(seq 1 24); do s=$(gh pr view 324 --json statusCheckRollup --jq
' [.statusCheckRollup[] | (.conclusion // .status)] | (map(select(.=="SUCCESS"))|length) as $ok |
"\($ok)/\($length)" 2>/dev/null); pend=$(gh pr view 324 --json statusCheckRollup --jq
' [.statusCheckRollup[] | select((.conclusion // "PENDING"))|test("SUCCESS|NEUTRAL|SKIPPED")|not)]
| length' 2>/dev/null); echo "checks ok=$s pending/other=$pend"; if ["$pend" = "0"]; then
break; fi; sleep 10; done
echo "=== final ==="; gh pr view 324 --json mergeable,mergeStateStatus,statusCheckRollup --jq
'{mergeable, mergeStateStatus, failures: [.statusCheckRollup[] |
select((.conclusion//""=="FAILURE") | .name)]}' 2>&1
[/external_agent_tool_call]

```

#### 266. assistant (2026-07-02T06:11:32.862Z)

```

[external_agent_tool_result]
checks ok=0/0 pending/other=0
=== final ===
{"failures":[], "mergeStateStatus": "DIRTY", "mergeable": "CONFLICTING"}
[/external_agent_tool_result]

```

#### 267. assistant (2026-07-02T06:11:32.862Z)

`#324` shows `CONFLICTING` ? unexpected, since I branched from `db74f9b` and only touched 2 files. Master may have moved again (sibling merge). Let me check:

#### 268. assistant (2026-07-02T06:11:32.862Z)

```

[external_agent_tool_call: Bash]
description: Diagnose the conflict
command: git fetch origin 2>&1 | tail -2
echo "=== origin/master HEAD now ==="; git log --oneline -3 origin/master
echo "=== my branch base ==="; git log --oneline -1 HEAD
echo "=== did master move past db74f9b? merge-base ==="; git merge-base --is-ancestor HEAD
origin/master && echo "HEAD is ancestor of master" || echo "master advanced beyond my base"
echo "=== files changed on master since my base that overlap mine ==="; git diff --name-only
db74f9b origin/master 2>&1 | grep -E 'QUALITY_ITERATIONS|test_served_gate_a' || echo "(no
overlap with my 2 files)"
echo "=== what changed on master since db74f9b ==="; git log --oneline db74f9b..origin/master
2>&1 | head
[/external_agent_tool_call]

```

#### 269. assistant (2026-07-02T06:11:32.862Z)

```

[external_agent_tool_result]
=== origin/master HEAD now ===
9b37269 docs(backlog): track golden-regression-fixtures DoD remainder (M4, P0 rename) + optional
01/02 (#323)
32f4ee6 test(eval): re-ground served Gate-A litmus + QUALITY_ITERATIONS on the 15-clip golden
corpus (#322)
db74f9b docs(data): mark DATA_IN_GCS $7b blocker #1 resolved by #319 (#321)
=== my branch base ===
5edd9f8 test(eval): re-ground served Gate-A litmus to the 15-clip golden corpus
=== did master move past db74f9b? merge-base ===
master advanced beyond my base
=== files changed on master since my base that overlap mine ===
docs/QUALITY_ITERATIONS.md
tests/test_served_gate_a_regression.py
=== what changed on master since db74f9b ===
9b37269 docs(backlog): track golden-regression-fixtures DoD remainder (M4, P0 rename) + optional
01/02 (#323)
32f4ee6 test(eval): re-ground served Gate-A litmus + QUALITY_ITERATIONS on the 15-clip golden
corpus (#322)
[/external_agent_tool_result]

```

#### 270. assistant (2026-07-02T06:11:32.862Z)

**\*\*Key finding:\*\*** PR **##322** (``32f4ee6` test(eval): re-ground served Gate-A litmus + QUALITY\_ITERATIONS on the 15-clip golden corpus`) **\*\*already landed the same fix\*\*** ? a parallel session/owner fixed master while I was working (the shared-checkout reality again). My #324 is now a **\*\*duplicate\*\*** and conflicts because it touches the same two files. Let me verify #322 actually fixes it correctly before closing mine:

#### 271. assistant (2026-07-02T06:11:32.862Z)

```
[external_agent_tool_call: Bash]
description: Inspect 322's pins on master
command: echo "=== #322's pinned values in the litmus (master) ==="; git show
origin/master:tests/test_served_gate_a_regression.py | grep -nE "approx\(|mean_delta_f1|0\.5 *
agg|15-clip|9-clip|1\.36|0\.79|1\.16|0\.64" | head -20
echo; echo "=== does master's litmus pass on the live corpus? ==="; git stash list >/dev/null
2>&1; tmpd=$(mktemp -d); git show origin/master:tests/test_served_gate_a_regression.py >
"$tmpd/t.py" 2>/dev/null; cp "$tmpd/t.py" /tmp/master_litmus.py 2>/dev/null; echo "(saved master
copy)"
[/external_agent_tool_call]
```

#### 272. assistant (2026-07-02T06:11:32.862Z)

```
[external_agent_tool_result]
=== #322's pinned values in the litmus (master) ===
18:SECOND re-grounding ? #236 pinned the 2026-06-18 9-clip snapshot, which the corpus has since
28:same gate is -F1-neutral (?F1 ? +0.03) while still cutting over-segmentation (1.36?0.79).
108: # Honest measured value (2026-06-22, present 15-clip multicourt golden set): mean served
109: # ?F1 ? -0.023 (was ? -0.034 on the 9-clip / ? -0.008 on the retired original-6). Band
admits
111: assert -0.06 <= agg["mean_delta_f1"] <= 0.03, agg
113: assert agg["mean_f1_on"] >= 0.5 * agg["mean_f1_off"] - 1e-9, agg
178: # Honest measured snapshot (2026-06-22, present 15-clip multicourt set): mean ?F1 ?
+0.03,
179: # over-seg 1.36 ? 0.79 (was -0.001 / 1.16 ? 0.64 on the 9-clip; +0.074 / 1.68 ? 1.01 on
the
181: assert mean_delta == pytest.approx(0.03, abs=0.03), mean_delta
182: assert mean_seg_off == pytest.approx(1.36, abs=0.10), mean_seg_off
183: assert mean_seg_on == pytest.approx(0.79, abs=0.10), mean_seg_on

=== does master's litmus pass on the live corpus? ===
(saved master copy)
[/external_agent_tool_result]
```

#### 273. assistant (2026-07-02T06:11:32.862Z)

#322's pins are **\*\*byte-identical to mine\*\*** (``approx(0.03)``, ``approx(1.36)``, ``approx(0.79)``, same bands) ? two independent re-measurements reached the same numbers, which strongly cross-validates the re-grounding. Master is already fixed by #322. Let me definitively confirm master's litmus passes on the live corpus, then clean up my redundant #324.

#### 274. assistant (2026-07-02T06:11:32.862Z)

```
[external_agent_tool_call: Bash]
description: Verify master's litmus passes
command: echo "=== run MASTER's exact litmus file against the live corpus ==="
git checkout origin/master -- tests/test_served_gate_a_regression.py 2>&1
python -m pytest tests/test_served_gate_a_regression.py -q 2>&1 | tail -4
echo "=== restore my branch's copy ==="
git checkout HEAD -- tests/test_served_gate_a_regression.py 2>&1
echo "restored"
[/external_agent_tool_call]
```

#### 275. assistant (2026-07-02T06:11:32.862Z)

```
[external_agent_tool_result]
```

```
=== run MASTER's exact litmus file against the live corpus ===
..... [100%]
5 passed in 19.76s
=== restore my branch's copy ===
restored
[/external_agent_tool_result]
```

#### 276. assistant (2026-07-02T06:11:32.862Z)

**\*\*Confirmed: master (with #322) passes the litmus ? 5 passed.\*\*** Master is already fixed; my #324 is a redundant duplicate that independently reached the identical pinned values. Let me close it cleanly and delete its branch.

#### 277. assistant (2026-07-02T06:11:32.862Z)

```
[external_agent_tool_call: Bash]
description: Close dup PR, delete branch
command: gh pr close 324 --comment "Closing as a **duplicate of #322** (already merged), which
made the identical 15-clip re-grounding of ``test_served_gate_a_regression.py`` +
`QUALITY_ITERATIONS.md`. A parallel session landed it while I was working on this one.

Independent cross-validation: this PR reached the **same** re-measured pins (`mean_seg_off`
1.36, `mean_seg_on` 0.79, proposal ?F1 ? +0.03, served ?F1 ? -0.023) and the same conclusion
(corpus grew 9?15 via #316; durable invariants held ? corpus change, not a code regression).
Verified `origin/master` (with #322) passes the litmus locally: ``5 passed``. No further
action needed." 2>&1 | tail -3
echo "=== delete remote branch ==="; git push origin --delete fix/served-gate-a-
regrounded-15clip 2>&1 | tail -2
echo "=== switch off the dead branch + delete locally ==="; git checkout fix/worker-train-dated-
label-join 2>&1 | tail -1; git branch -D fix/served-gate-a-regrounded-15clip 2>&1 | tail -1
[/external_agent_tool_call]
```

#### 278. assistant (2026-07-02T06:11:32.862Z)

```
[external_agent_tool_result]
? Closed pull request Khelsutra/badminton-highlight-indexer#324 (test(eval): re-ground served
Gate-A litmus to the 15-clip golden corpus)
=== delete remote branch ===
To https://github.com/Khelsutra/badminton-highlight-indexer.git
- [deleted] fix/served-gate-a-regrounded-15clip
=== switch off the dead branch + delete locally ===
error: pathspec 'fix/worker-train-dated-label-join' did not match any file(s) known to git
error: cannot delete branch 'fix/served-gate-a-regrounded-15clip' used by worktree at
'C:/Users/avidu/Projects/badminton-highlight-indexer'
[/external_agent_tool_result]
```

#### 279. assistant (2026-07-02T06:11:32.862Z)

#324 closed and its remote branch deleted. The local `fix/worker-train-dated-label-join` was already pruned by a sibling, so let me park on a detached `origin/master` (master itself is locked by another worktree) and drop the redundant local branch:

#### 280. assistant (2026-07-02T06:11:32.862Z)

```
[external_agent_tool_call: Bash]
description: Detach to master, delete redundant local branch
command: git checkout --detach origin/master 2>&1 | tail -2
git branch -D fix/served-gate-a-regrounded-15clip 2>&1 | tail -1
echo "=== where am I ==="; git log --oneline -1; git status -sb 2>&1 | head -1
[/external_agent_tool_call]
```

#### 281. assistant (2026-07-02T06:11:32.862Z)

```
[external_agent_tool_result]
```

HEAD is now at 9b37269 docs(backlog): track golden-regression-fixtures DoD remainder (M4, P0 rename) + optional 01/02 (#323)  
Deleted branch fix/served-gate-a-regrounded-15clip (was 5edd9f8).  
=== where am I ===  
9b37269 docs(backlog): track golden-regression-fixtures DoD remainder (M4, P0 rename) + optional 01/02 (#323)

## HEAD (no branch)

[/external\_agent\_tool\_result]

### 282. assistant (2026-07-02T06:11:32.862Z)

Now let me correct my memory/checkpoint notes ? the actual merged fix is **##322** (parallel session), not my closed-duplicate #324:

### 283. assistant (2026-07-02T06:11:32.862Z)

[external\_agent\_tool\_call: Edit]  
file: C:\Users\avidu\claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\golden-corpus-grew-9-to-15.md  
[/external\_agent\_tool\_call]

### 284. assistant (2026-07-02T06:11:32.863Z)

[external\_agent\_tool\_result]  
The file C:\Users\avidu\claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\golden-corpus-grew-9-to-15.md has been updated successfully. (file state is current in your context ? no need to Read it back)  
[/external\_agent\_tool\_result]

### 285. assistant (2026-07-02T06:11:32.863Z)

[external\_agent\_tool\_call: Edit]  
file: C:\Users\avidu\claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\MEMORY.md  
[/external\_agent\_tool\_call]

### 286. assistant (2026-07-02T06:11:32.863Z)

[external\_agent\_tool\_result]  
The file C:\Users\avidu\claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\MEMORY.md has been updated successfully. (file state is current in your context ? no need to Read it back)  
[/external\_agent\_tool\_result]

### 287. assistant (2026-07-02T06:11:32.863Z)

[external\_agent\_tool\_call: Edit]  
file: C:\Users\avidu\claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\current-state.md  
[/external\_agent\_tool\_call]

### 288. assistant (2026-07-02T06:11:32.863Z)

[external\_agent\_tool\_result]  
The file C:\Users\avidu\claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\current-state.md has been updated successfully. (file state is current in your context ? no need to Read it back)  
[/external\_agent\_tool\_result]

### 289. assistant (2026-07-02T06:11:32.863Z)

Everything is reconciled. Here's the complete picture.

## ? Both requested PRs merged

```
- **[#319](https://github.com/Khelsutra/badminton-highlight-indexer/pull/319)** ? `worker train`
dated-label join fix (`_label_clip_name` helper + 6 tests, worker coverage 82%?84%). Squash-
merged ? `b72e6de`.
- **[#321](https://github.com/Khelsutra/badminton-highlight-indexer/pull/321)** ? the doc fix:
`DATA_IN_GCS.md` §7b blocker #1 flipped to ? RESOLVED + §8 tracker row. Squash-merged ?
`db74f9b`.
```

Both passed a 3-dimension adversarial verification workflow (code-correctness, doc-accuracy/no-fabrication, test-rigor) ? `all\_ok: true`, zero blocking issues ? before I merged them.

## ? Why master was "broken" ? and it's already fixed

**Diagnosis:** not a code regression. Master's \*CI\* is green; only the \*local\* full suite failed, on one test:

```
test_served_gate_a_regression::test_proposal_windowing_gate_a_is_f1_neutral_and_cuts_over_seg
 assert mean_seg_off == 1.16 ± 0.10 -> got 1.36
...
```

That litmus pins **date-stamped over-segmentation snapshots** measured on a **9-clip** golden set, and **skips in CI** (its manifest is gitignored) ? so it can only break locally. The **#316** GCS migration restored the original-6 single-court labels, growing the local corpus **9 ? 15 clips**. Single-court clips over-segment, so the blend moved `mean_seg_off` to 1.36. The litmus tripped **exactly as designed** ("snapshots will need re-measurement as the corpus grows"). I confirmed every  **durable**  invariant still holds ? corpus change, not a bug.

**The fix was already merged in parallel.** While I was re-measuring and re-grounding it, another session landed the **identical** fix as **[#322](https://github.com/Khelsutra/badminton-highlight-indexer/pull/322)** (`32f4ee6`). My independent re-measurement reached the **same** pins (`mean_seg_off` 1.36, `mean_seg_on` 0.79, proposal ?F1 ? +0.03, served ?F1 ? ?0.023) ? a nice cross-validation. So I **closed my duplicate** **[#324]** and deleted its branch. I verified `origin/master` (now `9b37269`, includes #322) passes the litmus locally: **5 passed**.

**Master is green** ? both broken thing (the litmus) and the worker-train bug are resolved.

## Notes

- A genuine finding surfaced: with the original-6 back (they carried the archived **+0.074** Gate-A win), the proposal-windowing gate is now mildly **F1-positive** again, while the served operating point stays mildly negative. The corpus 9?15 growth likely affects other eval narratives ? worth a wider sweep sometime.
- I left a re-usable measurement helper at `scratch/measure_served_gate_a_litmus` (gitignored) for re-grounding this litmus next time the corpus changes.
- Recorded the corpus-state change as a memory so a future session hitting this litmus failure checks corpus size before suspecting a regression.

One housekeeping note from the shared checkout: this repo has `master` checked out in another worktree (`tier1-windowing`) and several sibling sessions active ? that's how #322 landed underneath me. I branched everything from `origin/master` and staged explicit paths throughout, so nothing collided.

## 290. user (2026-07-02T06:11:32.863Z)

so no pending items for you except the wider sweep which can be done after more iterations?

## 291. assistant (2026-07-02T06:11:32.863Z)

Good question ? let me not just agree; let me verify it on the actual current master before confirming. I'll run the full suite on `9b37269` (where I'm parked) and scan for any **other** eval tests/docs still pinned to the old 9-clip corpus.

## 292. assistant (2026-07-02T06:11:32.863Z)

[external\_agent\_tool\_call: Bash]

description: Confirm HEAD then note suite launch  
command: git log --oneline -1; echo "=== launch full suite on current master ==="  
[/external\_agent\_tool\_call]

**293. assistant (2026-07-02T06:11:32.863Z)**

[external\_agent\_tool\_result]  
9b37269 docs(backlog): track golden-regression-fixtures DoD remainder (M4, P0 rename) + optional  
01/02 (#323)  
=== launch full suite on current master ===  
[/external\_agent\_tool\_result]

**294. assistant (2026-07-02T06:11:32.863Z)**

[external\_agent\_tool\_call: Bash]  
description: Full suite on current master  
command: python run\_all\_tests.py 2>&1 | tail -6  
[/external\_agent\_tool\_call]

**295. assistant (2026-07-02T06:11:32.863Z)**

[external\_agent\_tool\_result]  
Command running in background with ID: bq01qap6h. Output is being written to:  
C:\Users\avidu\AppData\Local\Temp\claude\C--Users\avidu-Projects-badminton-highlight-  
indexer\73a3938e-1da3-4e28-bfa7-346a15718301\tasks\bq01qap6h.output. You will be notified when  
it completes. To check interim output, use Read on that file path.  
[/external\_agent\_tool\_result]

**296. assistant (2026-07-02T06:11:32.863Z)**

Now scan for any \*other\* eval tests/docs still pinned to the old 9-clip corpus (the scope of the  
"wider sweep"):

**297. assistant (2026-07-02T06:11:32.863Z)**

[external\_agent\_tool\_call: Grep]  
input: {"pattern":"9 multicourt|9-clip|retired original-6|original-6.\*retired|present  
9|1\\.16|1\\.68|0\\.64","glob":"\*\*/\*.py,md"},"output\_mode":"files\_with\_matches"}  
[/external\_agent\_tool\_call]

**298. assistant (2026-07-02T06:11:32.863Z)**

[external\_agent\_tool\_result]  
Found 8 files  
docs\QUALITY\_ITERATIONS.md  
tests\test\_served\_gate\_a\_regression.py  
docs\ABLATION\_LAYER.md  
tests\test\_ablation.py  
docs\archives\quality-iterations\README.md  
tests\test\_per\_user\_gate.py  
docs\archives\quality-iterations\2026-06-first-ab-experiment\README.md  
docs\archives\quality-iterations\2026-06-multivideo-lovo\README.md  
[/external\_agent\_tool\_result]

**299. assistant (2026-07-02T06:11:32.863Z)**

Let me characterize those hits ? are `test\_ablation.py` / `test\_per\_user\_gate.py` corpus-pinned  
(would break on 15 clips), and is `ABLATION\_LAYER.md` carrying the stale 9-clip narrative?

**300. assistant (2026-07-02T06:11:32.863Z)**

[external\_agent\_tool\_call: Bash]  
description: Characterize the other hits

```

command: echo "=== test_ablation.py: skip-gating + any 9/1.16 pins ==="; grep -nE
"skipif|importorskip|isfile|1\.16|1\.68|0\.64|9 |9-clip|approx\(|MANIFEST|human_lovo|golden"
tests/test_ablation.py 2>&1 | head -25
echo; echo "=== test_per_user_gate.py: the matched lines ==="; grep -nE "9
multicourt|9-clip|original-6|1\.16|1\.68|0\.64|approx\(" tests/test_per_user_gate.py 2>&1 | head
-15
echo; echo "=== ABLATION_LAYER.md: the matched lines (context) ==="; grep -nE "9
multicourt|9-clip|retired|original-6|1\.16|1\.68|0\.64" docs/ABLATION_LAYER.md 2>&1 | head -15
[/external_agent_tool_call]

```

### 301. assistant (2026-07-02T06:11:32.863Z)

```

[external_agent_tool_result]
=== test_ablation.py: skip-gating + any 9/1.16 pins ===
13: - the golden litmus reproduction ? on the 6 real ``output/*_golden_features.json`` the
runner
15: cleanly when the golden JSONs are absent (CI), runs when present (owner box).
205: assert pytest.approx(r_non.delta_f1) == r_struct.delta_f1
221: assert abs(row.delta_f1) < 1e-9 # honestly a wash
252: assert r.ci_low - 1e-9 <= r.delta_f1 <= r.ci_high + 1e-9
324: pytest.importorskip("reportlab")
325: pypdf = pytest.importorskip("pypdf")
332: assert os.path.isfile(out)
353: with open(tmp_path / "noaudio_golden_features.json", "w", encoding="utf-8") as f:
361: """Write a LEANER velocity-windowing/distill ``*_golden_features.json`` ? the schema the
369: with open(os.path.join(d, f"{name}_golden_features.json"), "w", encoding="utf-8") as f:
390:# ----- the golden
litmus
396: def _golden_present() -> bool:
397: """True iff ?6 FULL-FUSION-SCHEMA golden files exist (the litmus needs the
shuttle/person cue
402: for p in glob.glob(os.path.join(GOLDEN_DIR, "*_golden_features.json")):
413:@pytest.mark.skipif(not _golden_present(),
414: reason="golden output/*_golden_features.json absent (run fusion_golden
first)")
416: """LITMUS: on the 6 real golden feature JSONs the runner reproduces the hand-measured
Gate-A
419: where the golden substrate is absent; runs on the owner box where it is present."""
428: assert row.delta_f1 == pytest.approx(0.074, abs=0.005)
430: assert row.sign_p_value == pytest.approx(0.031, abs=0.005)
431: assert row.ci_low == pytest.approx(0.042, abs=0.01)
432: assert row.ci_high == pytest.approx(0.104, abs=0.01)
436: # C4: over-segmentation drops 1.68 ? 1.01 when Gate-A is on (full).
437: assert row.seg_ratio_full == pytest.approx(1.01, abs=0.05)

=== test_per_user_gate.py: the matched lines ===
67: a = [0.60, 0.62, 0.65, 0.61, 0.63, 0.64, 0.66, 0.62]
83: a = [0.60, 0.62, 0.65, 0.61, 0.63, 0.64, 0.66, 0.62]

=== ABLATION_LAYER.md: the matched lines (context) ===
73: | over-seg ratio | 1.68 ? 1.01 | 1.68 ? 1.01 |
[/external_agent_tool_result]

```

### 302. assistant (2026-07-02T06:11:32.863Z)

Let me wait for the full-suite result on master (the authoritative check), then answer precisely:

### 303. assistant (2026-07-02T06:11:32.863Z)

```

[external_agent_tool_call: Bash]
description: Wait for full suite result on master
command: for i in $(seq 1 36); do if grep -q "RESULT: \\|passed\\|failed"
"C:\Users\avidu\AppData\Local\Temp\claud\C--Users-avidu-Projects-badminton-highlight-

```

```
indexer\73a3938e-1da3-4e28-bfa7-346a15718301\tasks\bq01qap6h.output" 2>/dev/null; then break;
fi; sleep 5; done
tail -6 "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\73a3938e-1da3-4e28-bfa7-346a15718301\tasks\bq01qap6h.output" 2>&1
[/external_agent_tool_call]
```

### 304. assistant (2026-07-02T06:11:32.863Z)

```
[external_agent_tool_result]
..... [95%]
..... [100%]
1430 passed in 98.85s (0:01:38)
=====
RESULT: PASS - all tests green
=====
[/external_agent_tool_result]
```

### 305. assistant (2026-07-02T06:11:32.864Z)

Correct ? confirmed with evidence. Full suite on current master (`9b37269`) is **all green: 1430 passed**, and CI is green too.

**Nothing pending for me from this work:**

- [#319](https://github.com/Khelsutra/badminton-highlight-indexer/pull/319) (worker-train fix) + [#321](https://github.com/Khelsutra/badminton-highlight-indexer/pull/321) (doc) ? merged.
- The local litmus break ? fixed by [#322](https://github.com/Khelsutra/badminton-highlight-indexer/pull/322) (parallel session); my duplicate #324 closed; no open PRs or uncommitted work left by me.

**One honest correction on the "wider sweep" ?** I overstated it last turn. Having now scanned the other eval tests/docs, it's narrower than "likely affects other narratives," and may be a non-issue:

- I checked the other corpus-pinned litmus, `tests/test\_ablation.py` (+ `docs/ABLATION\_LAYER.md`). It pins the original-6 **+0.074 / over-seg 1.68?1.01** on a **separate** substrate (`output/\*\_golden\_features.json`), **not** the `human\_lovo\_manifest.json` that grew to 15 ? so the 9?15 change doesn't touch it, and it's green.
- The other grep hits were either historical/archived records (correct as-is) or false positives (`test\_per\_user\_gate.py`'s `0.64` is synthetic test data, not a corpus number).

So there's **no known-stale eval item remaining** ? the served litmus was the only thing the corpus growth broke, and it's fixed. The "wider sweep" is now just an **optional, low-priority** sanity pass (e.g., if the owner later wants to re-derive any original-6-vs-blend comparisons now that the original-6 are back in the corpus). It can absolutely wait for more iterations, and it may well turn out to be a no-op.

For completeness, the genuinely-open items in this area are **not mine and pre-date this task**: `DATA\_IN\_GCS.md` \$7b blockers #2?#5 (Phase-3 eval-on-GCS plumbing, `manifests/golden.json` having no consumer, GPU-quota/secrets gating, promoting the `scratch/` migration helpers) and the nightly-regression owner deploy steps ? all owner-gated or separately tracked.